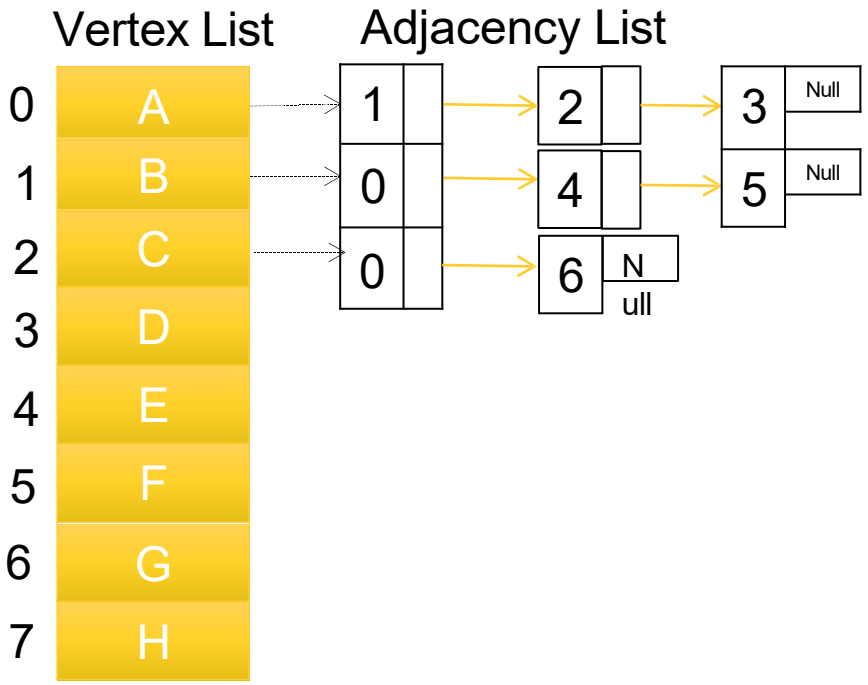
COMP 511

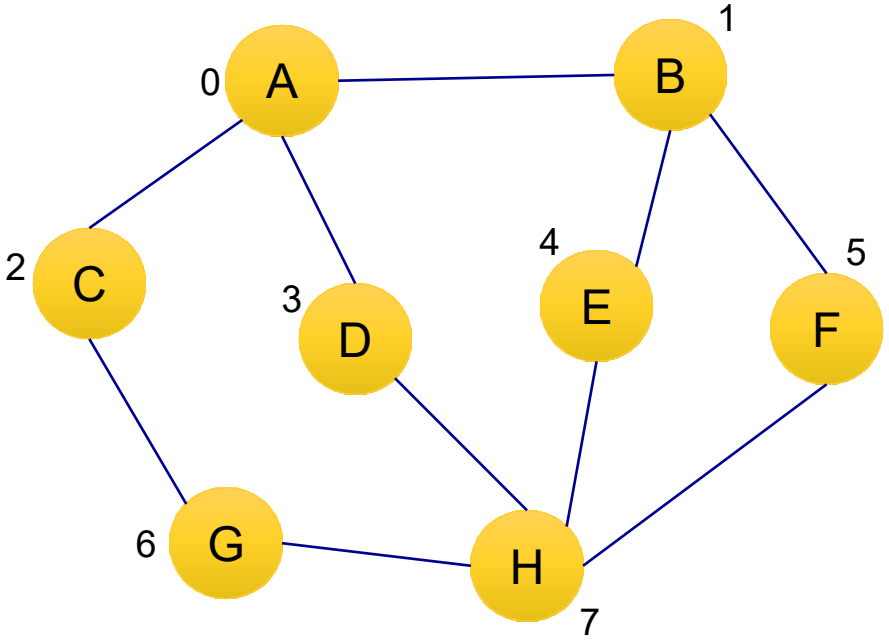
Computer Programming Foundations II

Graphs (cont'd)

Depth-First Search (DFS)

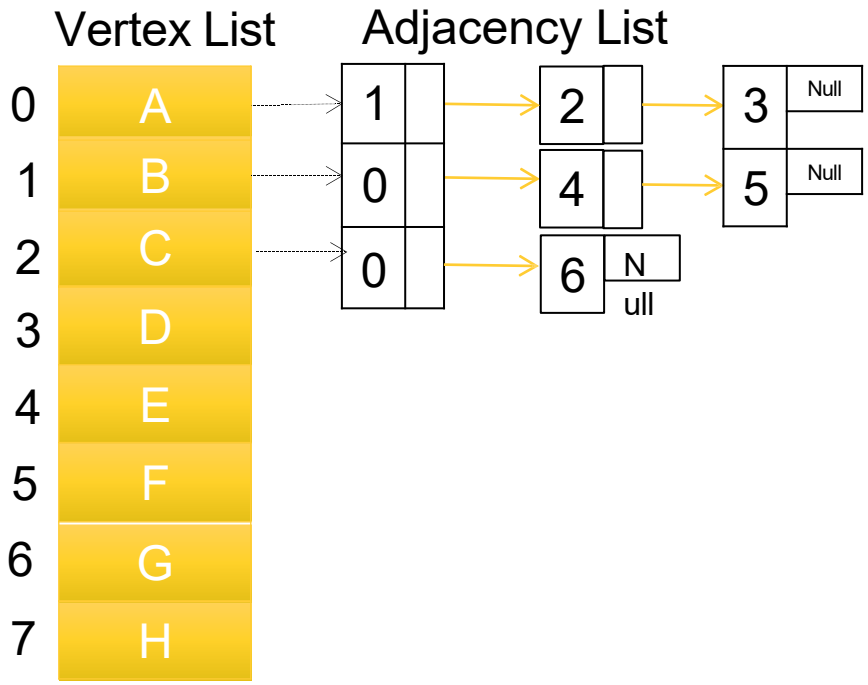
In a traversal, every vertex and every edge is examined.

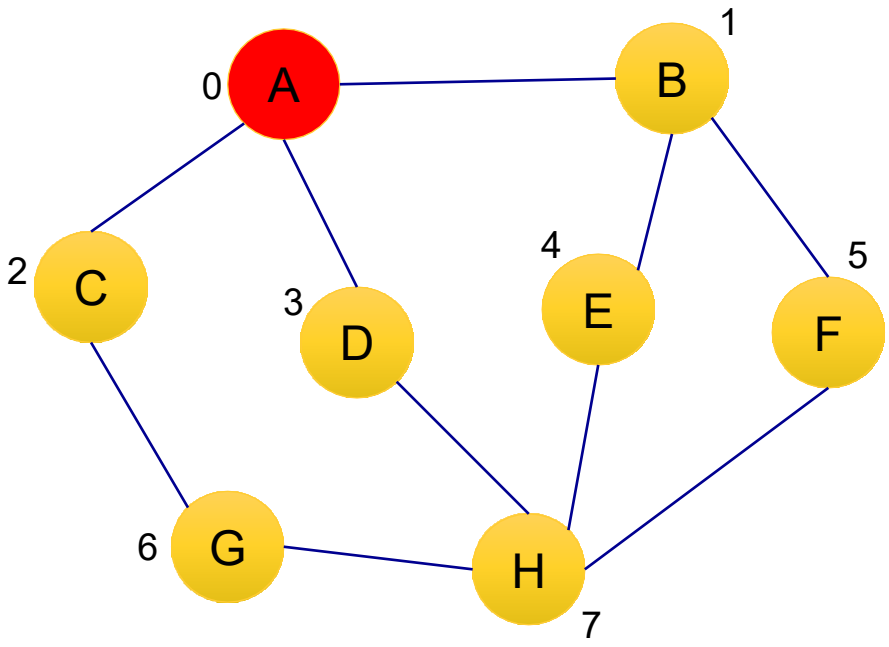




For DFS we use a **stack**

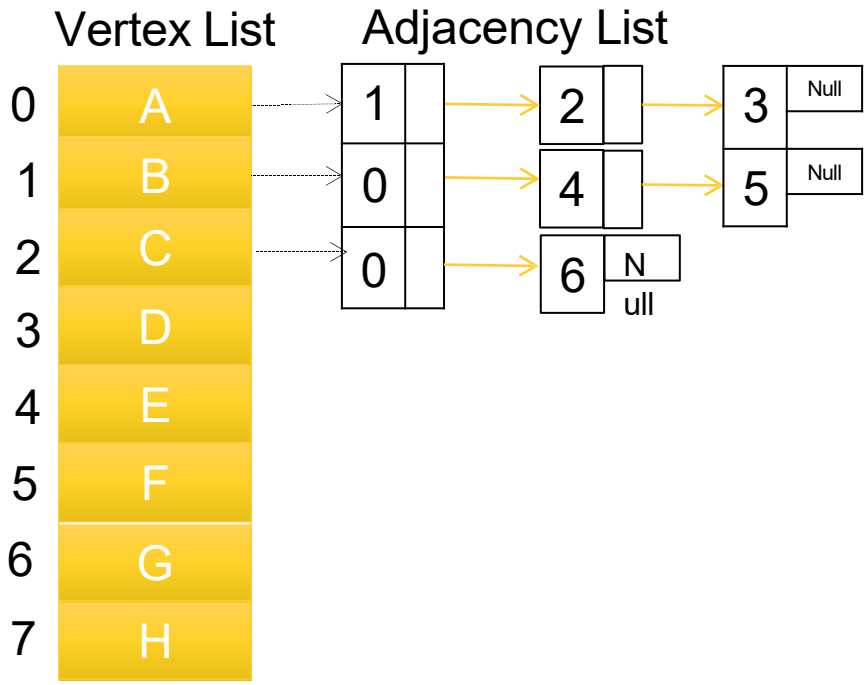
Depth-First Search (DFS)

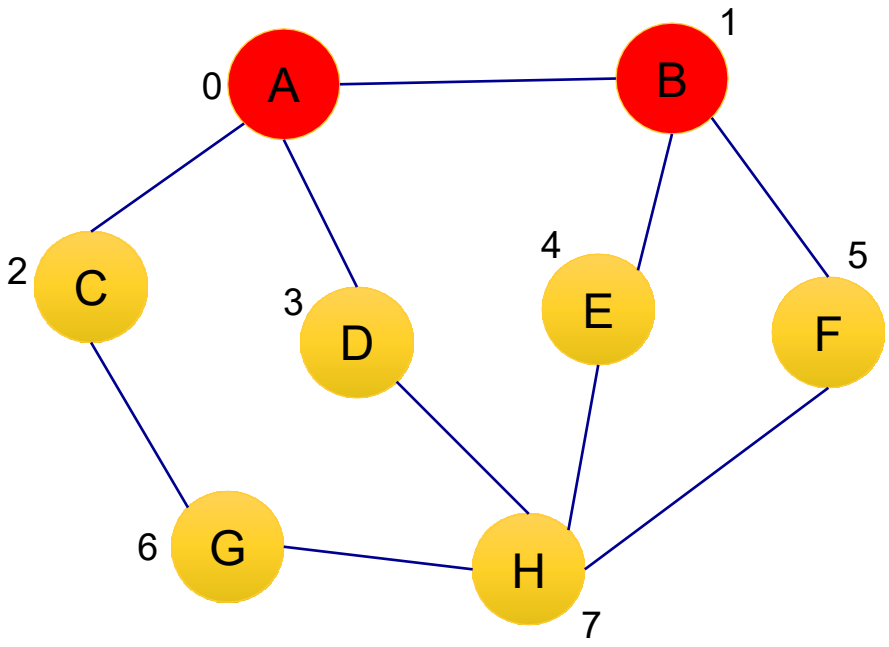




stack: push A (mark A as “visited”)

Depth-First Search (DFS)



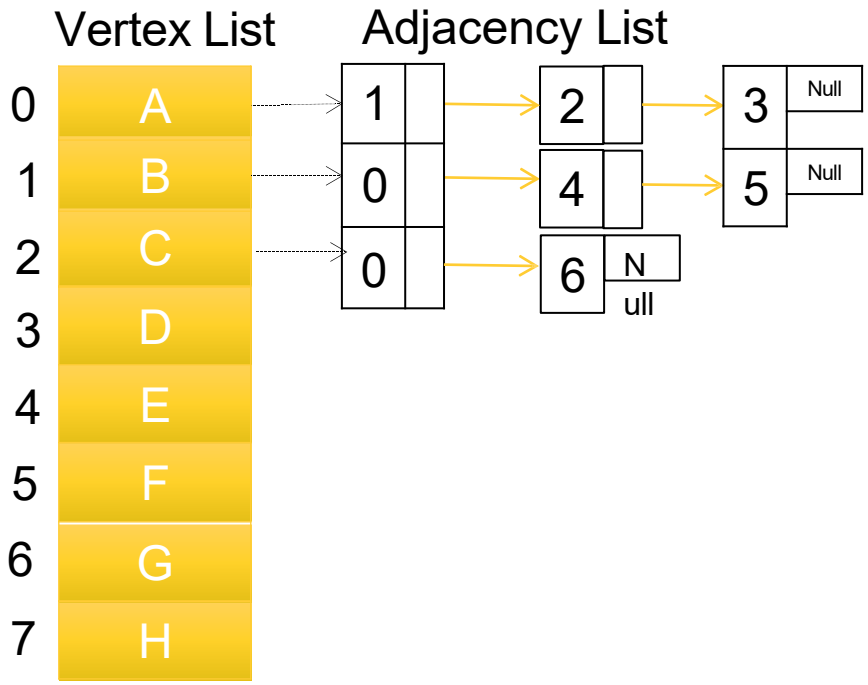


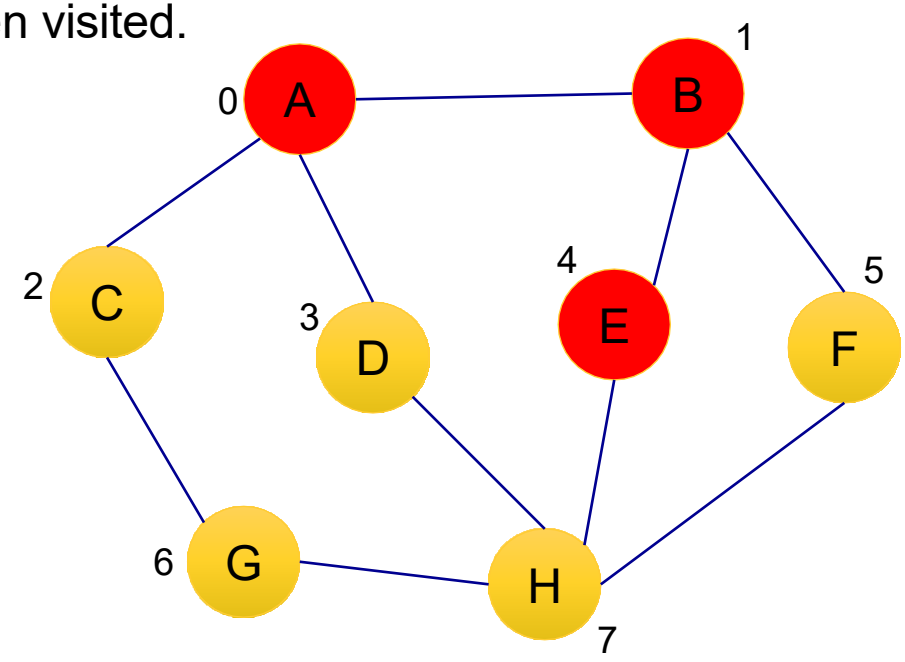
Push B (mark B as “visited”)

stack: A

Depth-First Search (DFS)

Repeat the procedure for B:
View all indices that have not yet been visited.





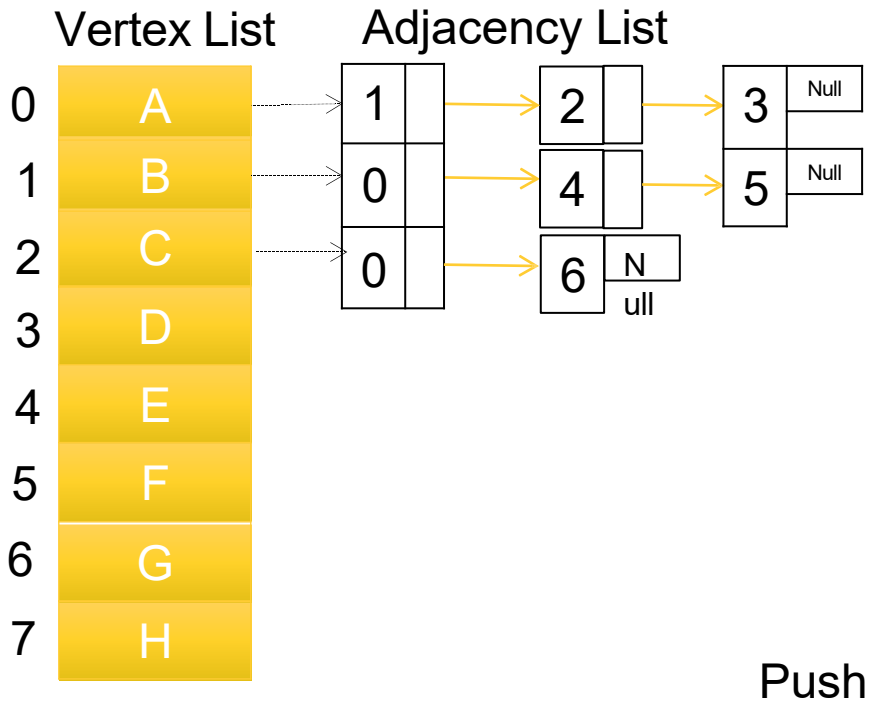
Push E (mark E as “visited”)

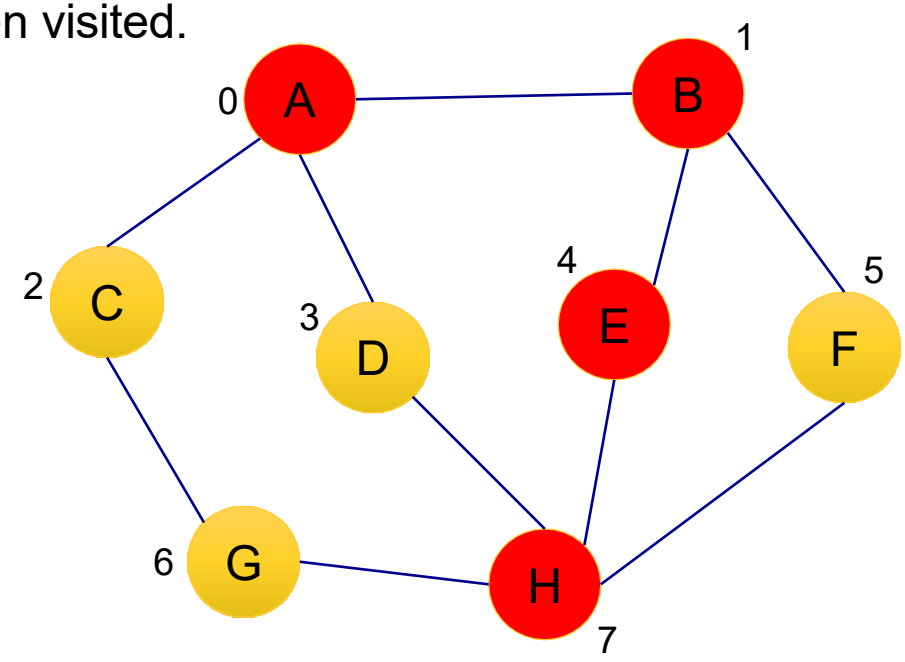
B

stack: A

Depth-First Search (DFS)

Repeat the procedure for E:
View all indices that have not yet been visited.





Push H (mark H as “visited”)

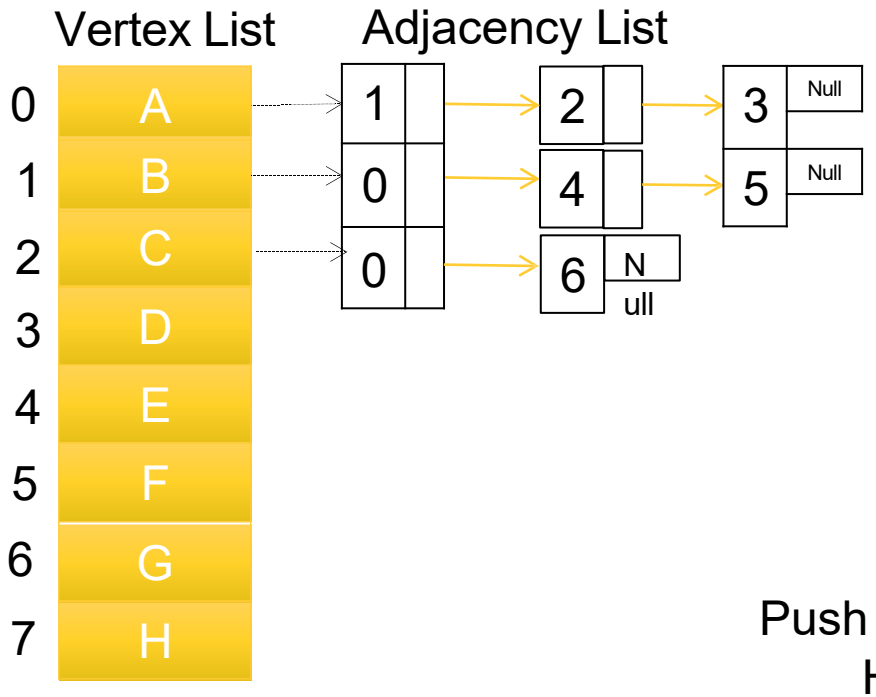
E

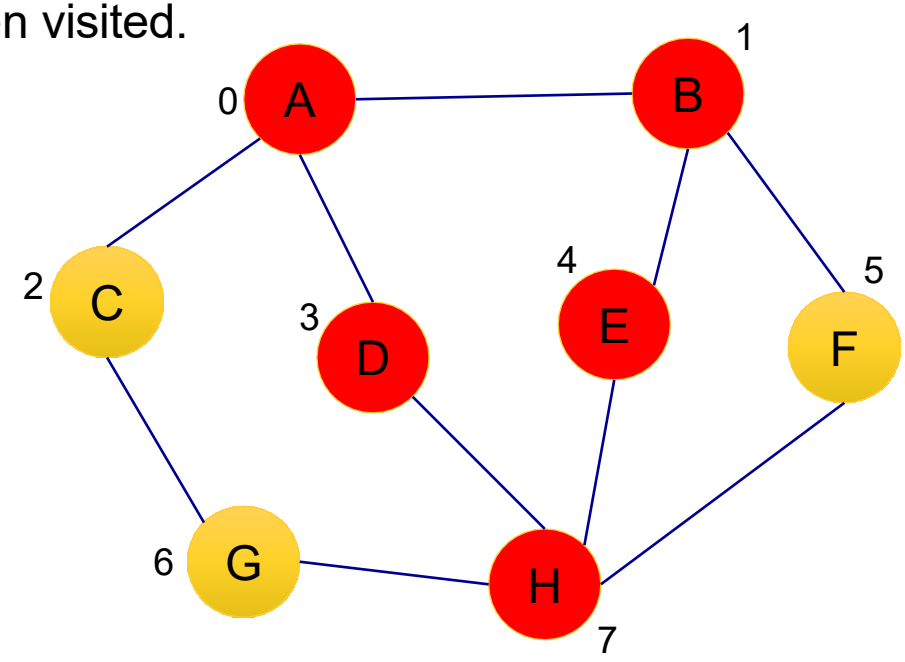
B

stack: A

Depth-First Search (DFS)

Repeat the procedure for H:
View all indices that have not yet been visited.





Push D (mark D as “visited”)

H

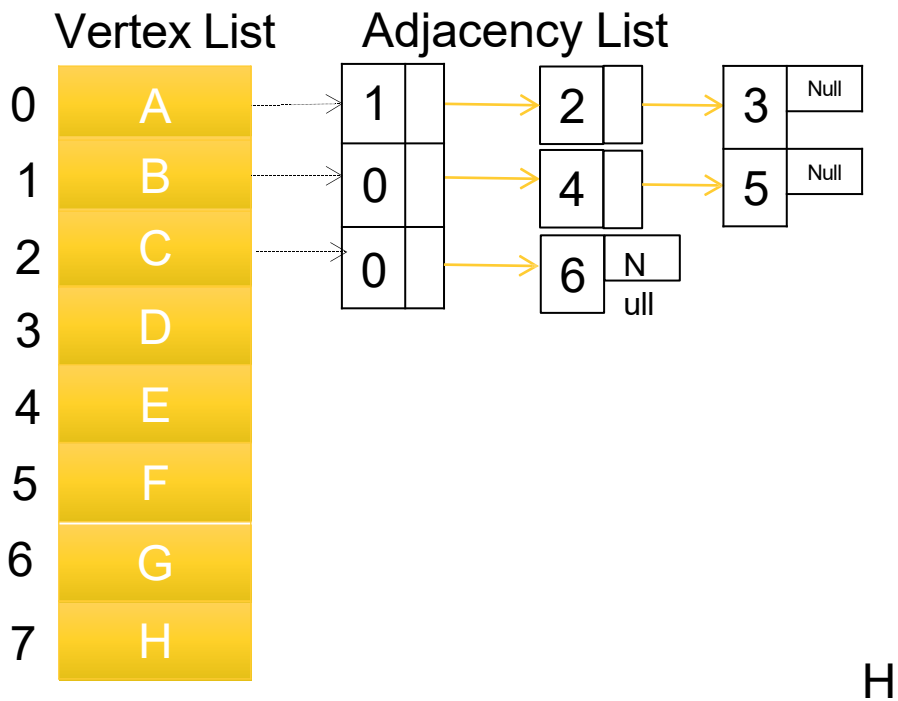
E

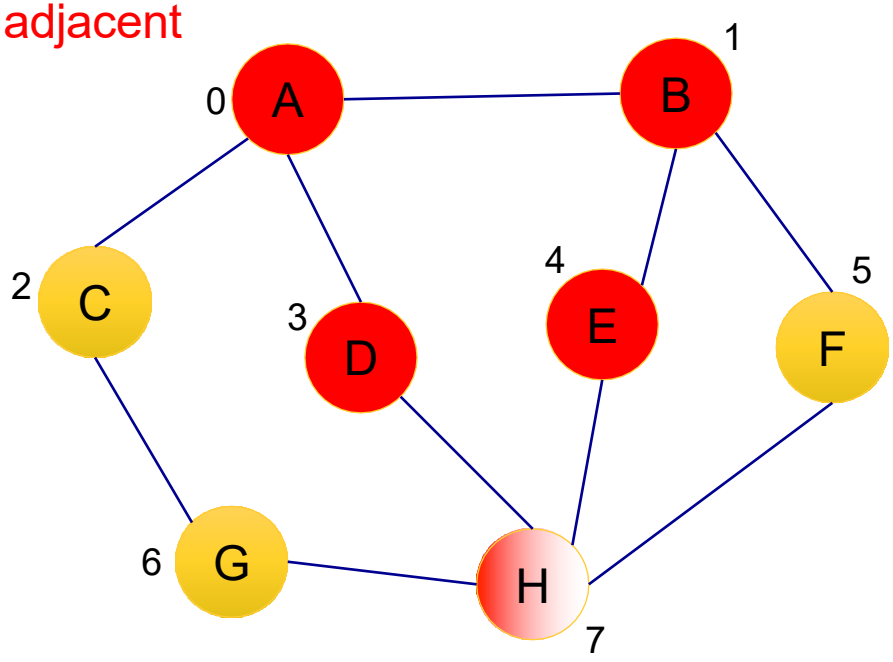
B

stack: A

Depth-First Search (DFS)

Repeat the procedure for D:
Since there are no un-visited vertices adjacent to D, we pop D off of the stack.

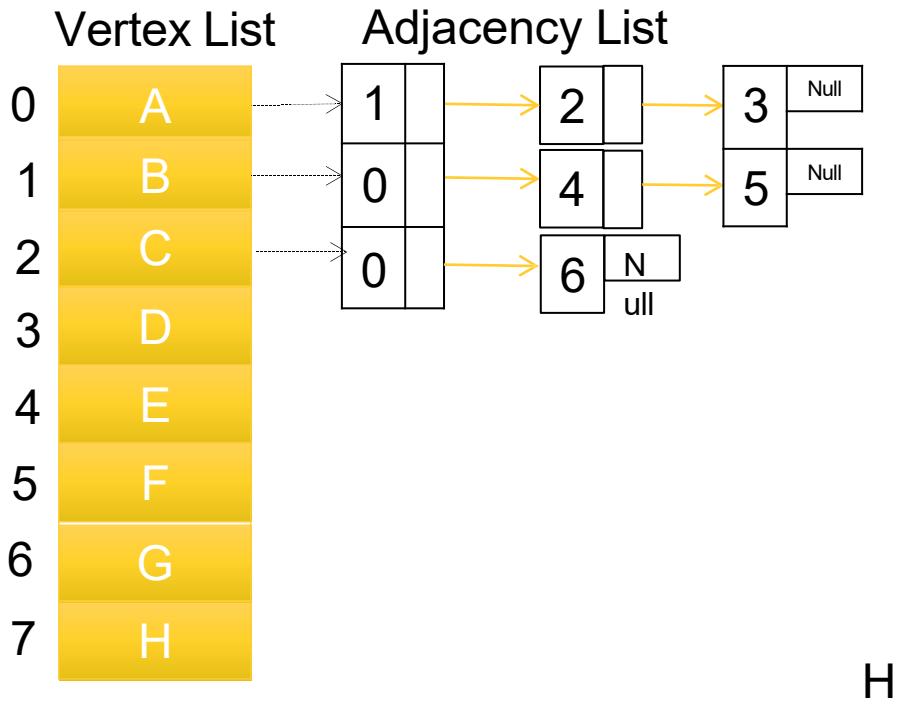


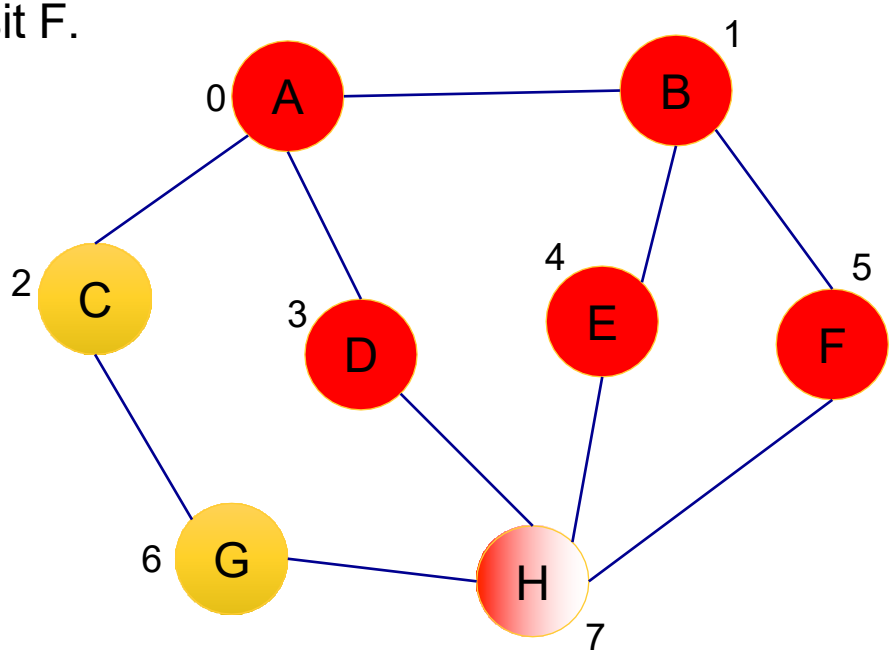


H
E
B
stack: A

Depth-First Search (DFS)

This takes us back to H. Does H have any Un-visited vertices? Yes – we next visit F.

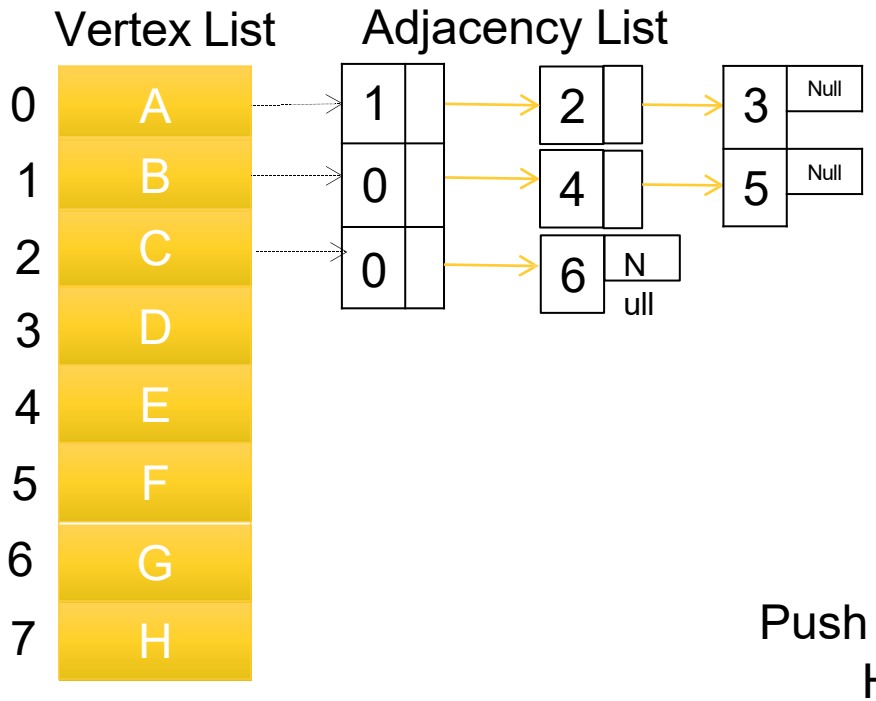


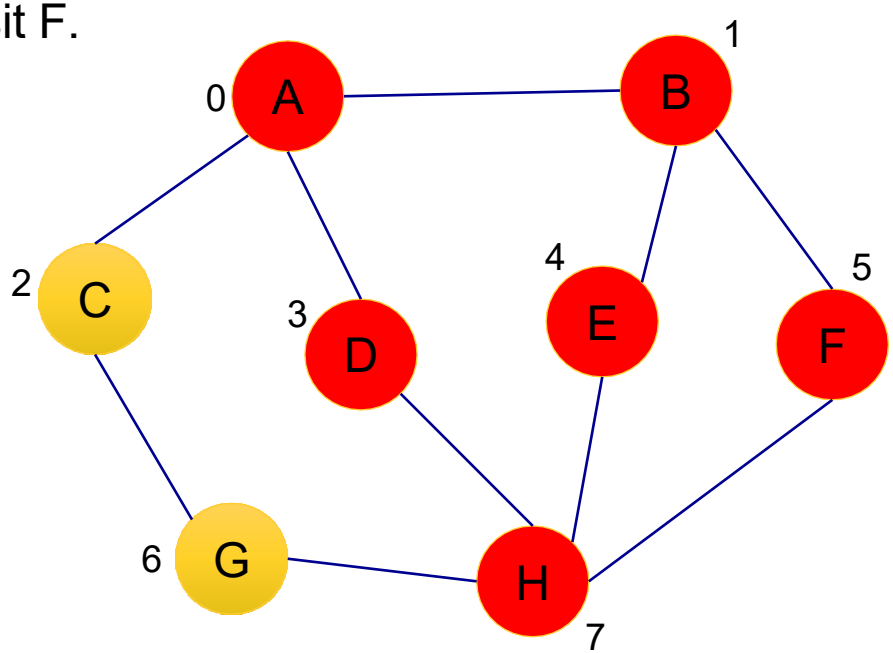


H
E
B
stack: A

Depth-First Search (DFS)

This takes us back to H. Does H have any Un-visited vertices? Yes – we next visit F.





Push F (mark F as "visited")

H

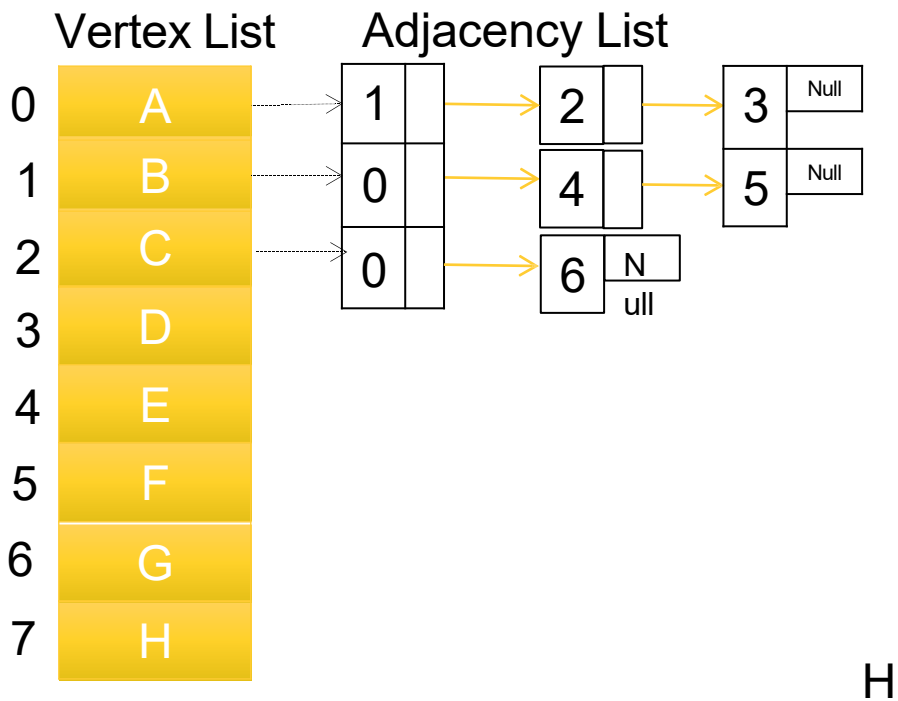
E

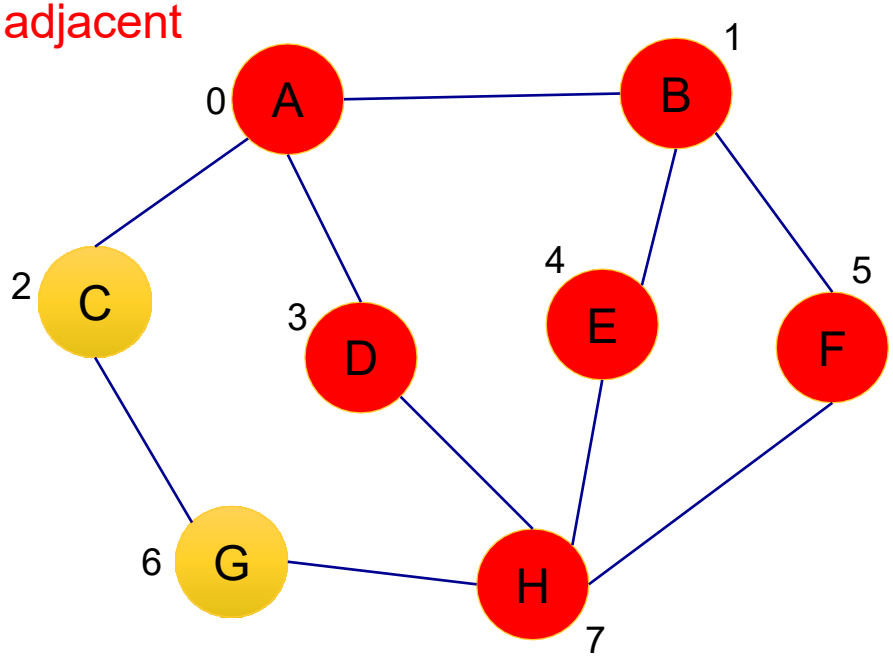
B

stack: A

Depth-First Search (DFS)

Repeat the procedure for F:
Since there are no un-visited vertices adjacent
To F, we pop F off of the stack.

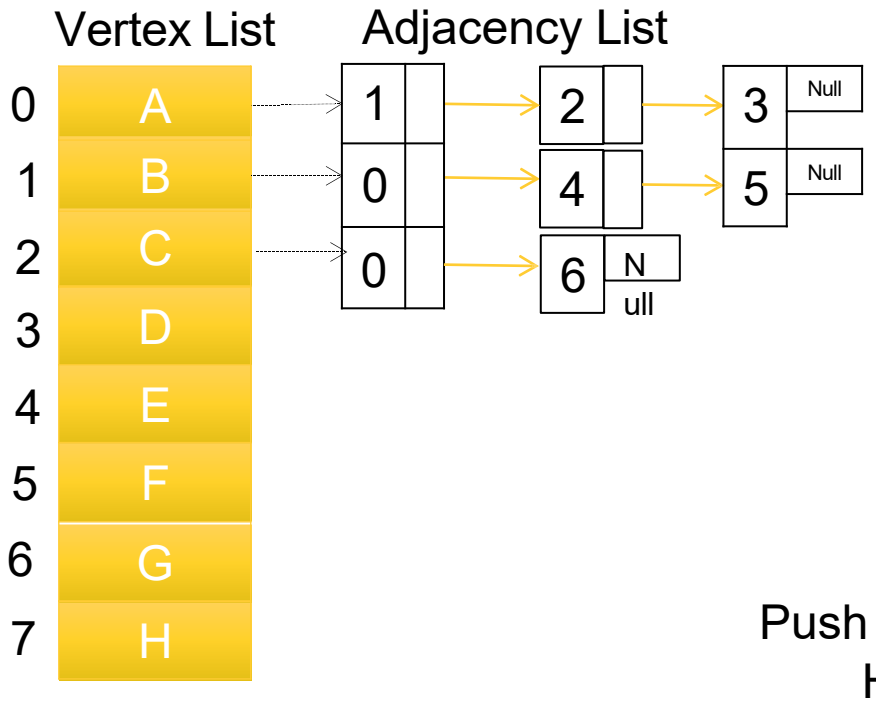


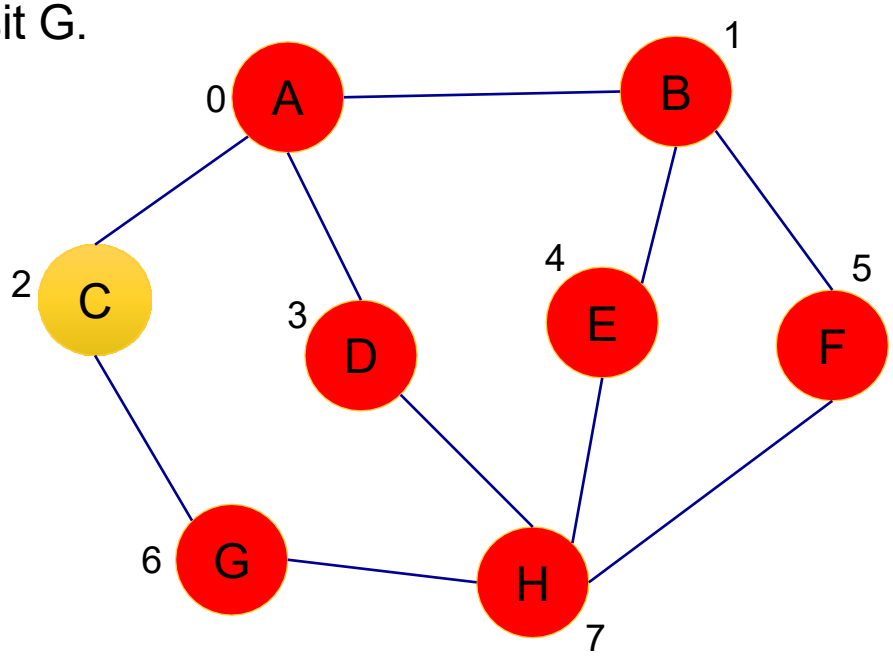


H
E
B
stack: A

Depth-First Search (DFS)

This takes us back to H. Does H have any Un-visited vertices? Yes – we next visit G.





Push G (mark G as "visited")

H

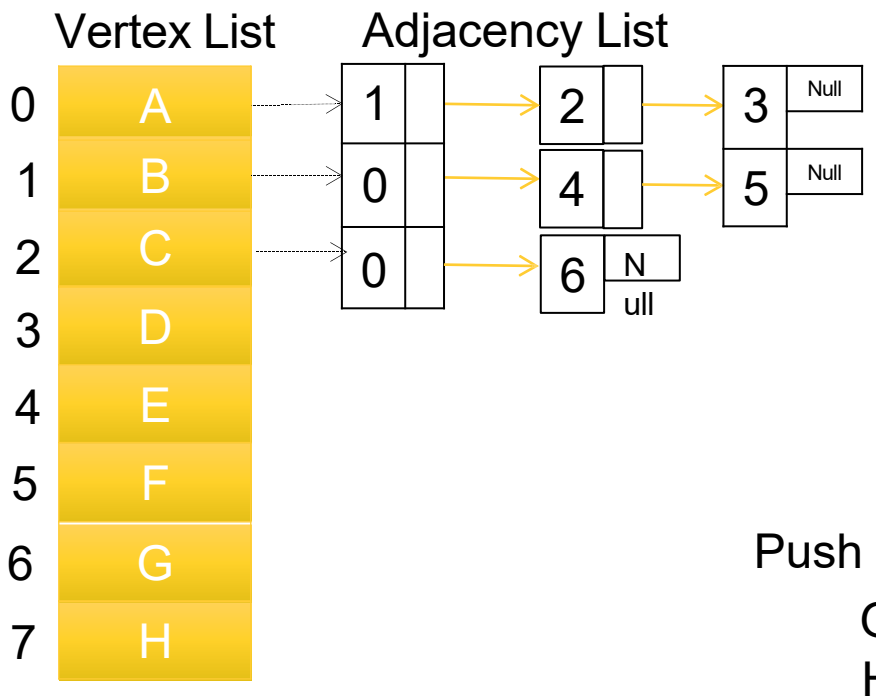
E

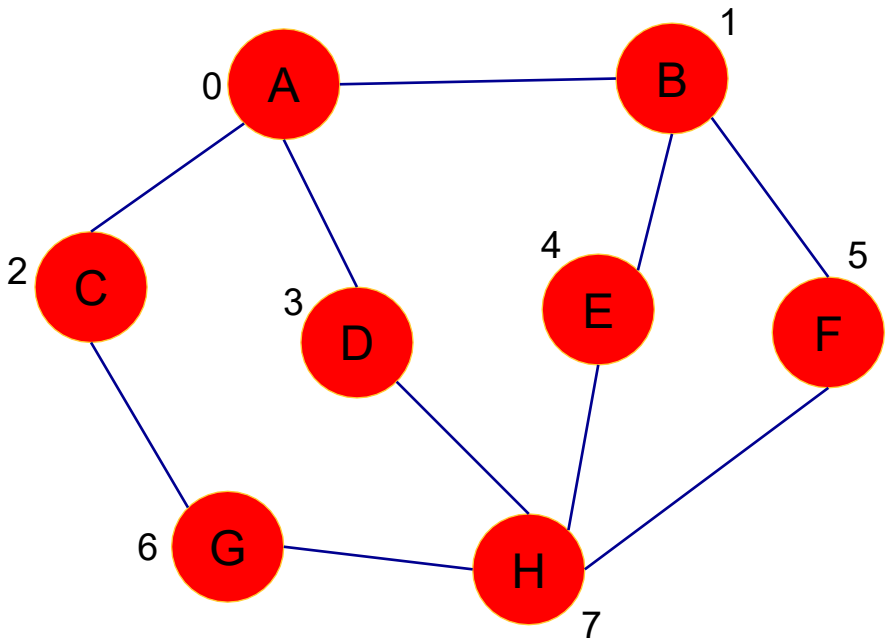
B

stack: A

Depth-First Search (DFS)

Repeat the procedure for G:
G has one remaining un-visited adjacent vertex: C





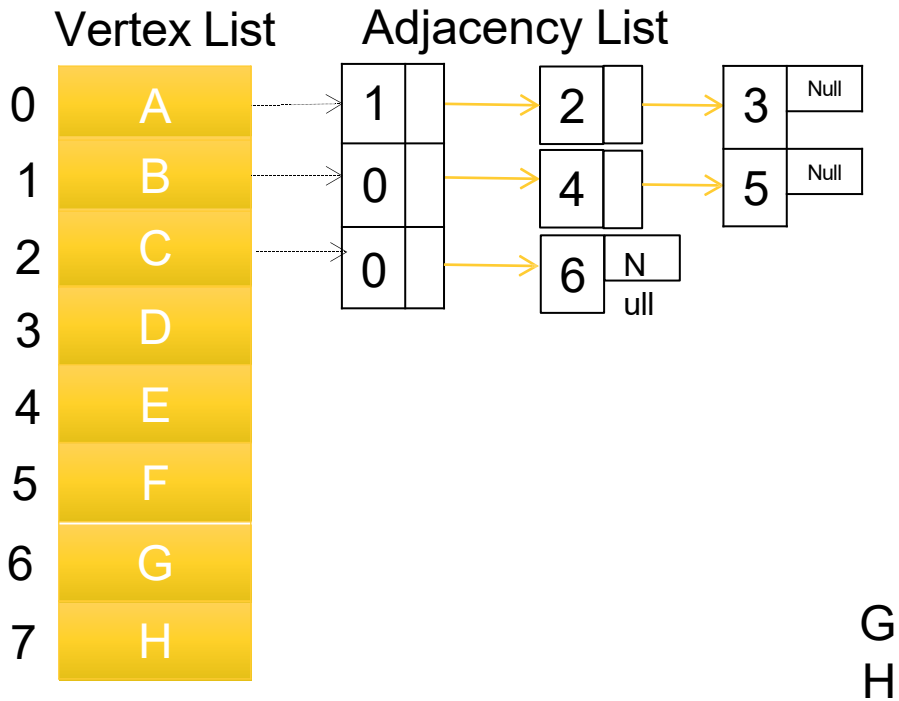
Push C (mark C as “visited”)

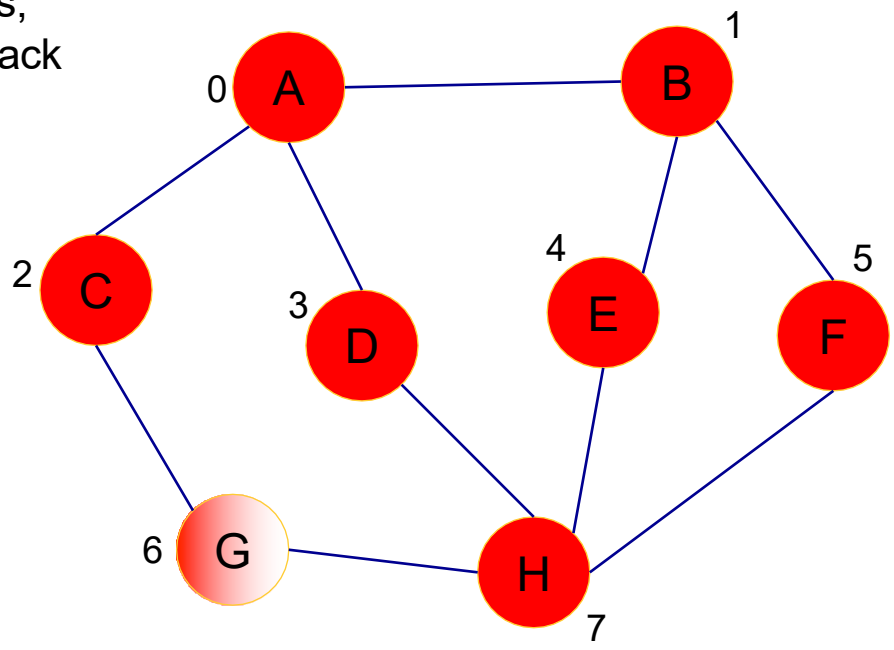
G
H
E
B

stack: A

Depth-First Search (DFS)

Repeat the procedure for C:
Since C has no un-visited adjacent vertices, we pop C off of the stack. This brings us back to G.

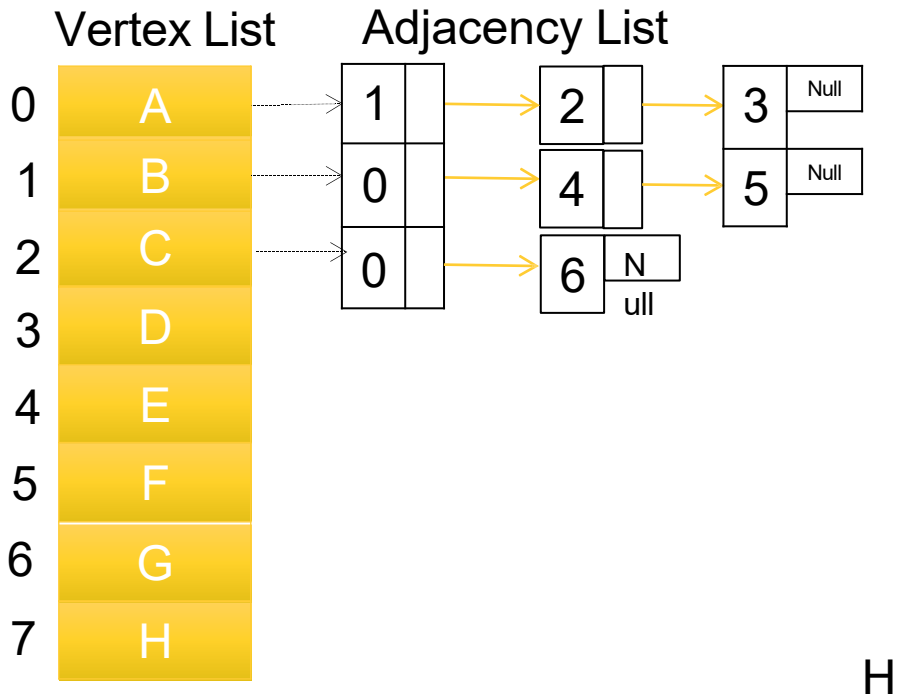


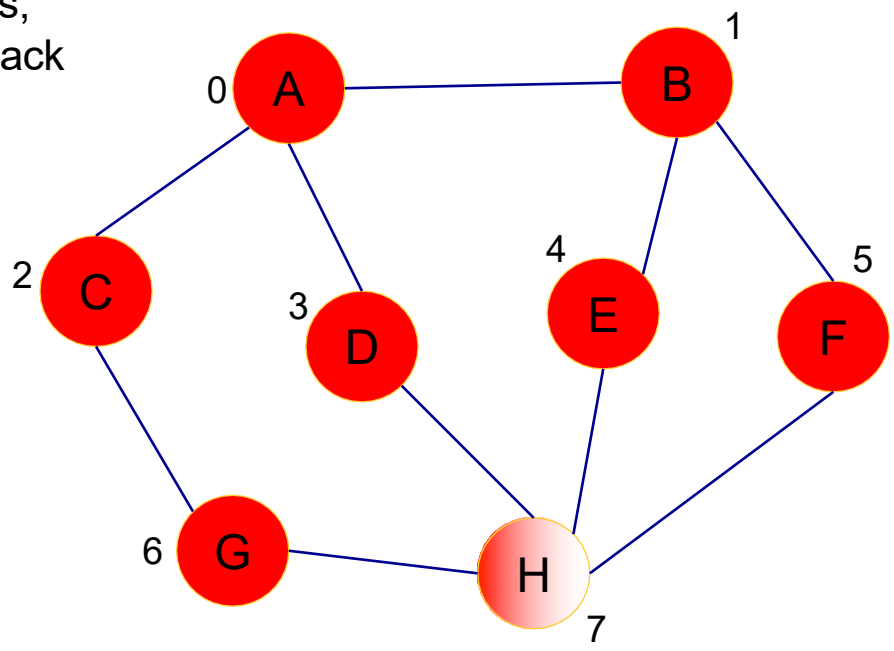


G
H
E
B
stack: A

Depth-First Search (DFS)

Repeat the procedure for G:
Since G has no un-visited adjacent vertices, we pop G off of the stack. This brings us back To H.

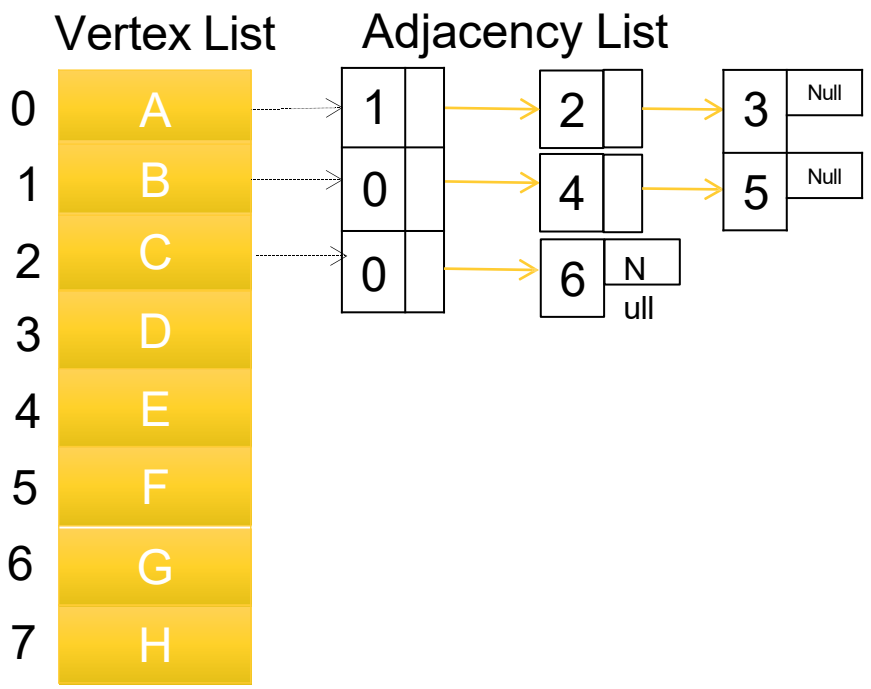


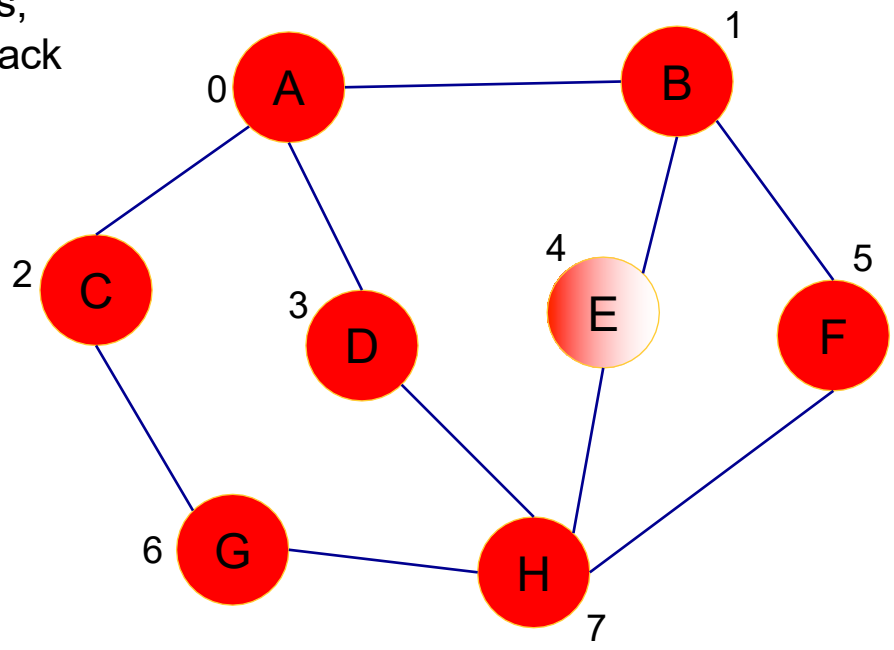


H
E
B
stack: A

Depth-First Search (DFS)

Repeat the procedure for H:
Since H has no un-visited adjacent vertices, we pop H off of the stack. This brings us back To E.





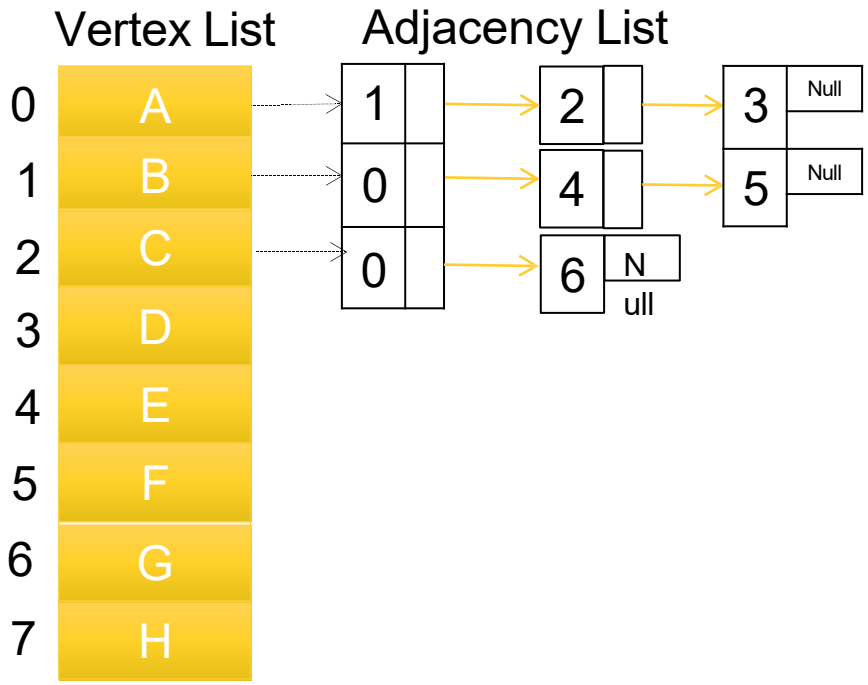
E

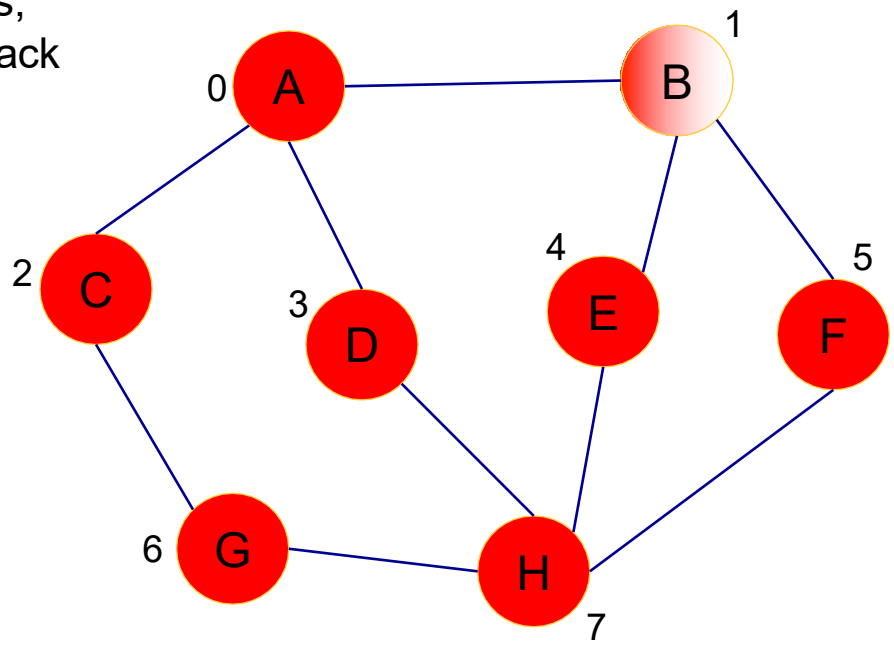
B

stack: A

Depth-First Search (DFS)

Repeat the procedure for E:
Since E has no un-visited adjacent vertices, we pop E off of the stack. This brings us back To B.



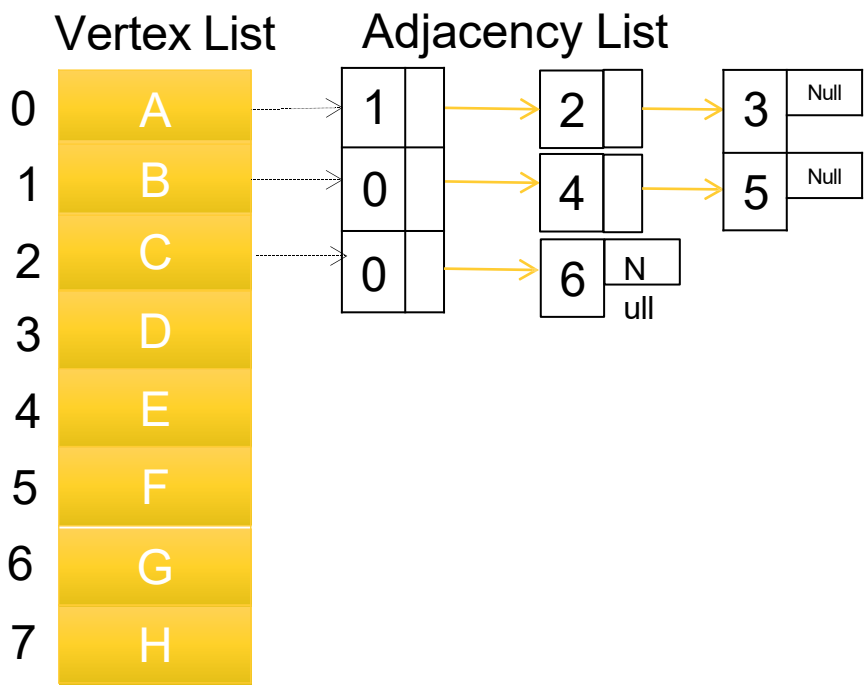


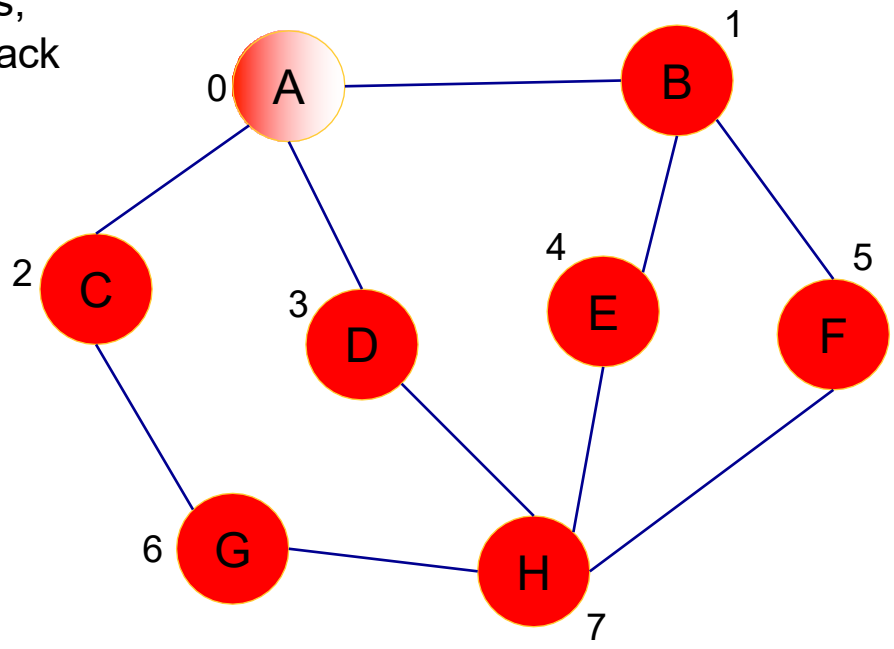
B

stack: A

Depth-First Search (DFS)

Repeat the procedure for B:
Since B has no un-visited adjacent vertices, we pop B off of the stack. This brings us back To A.

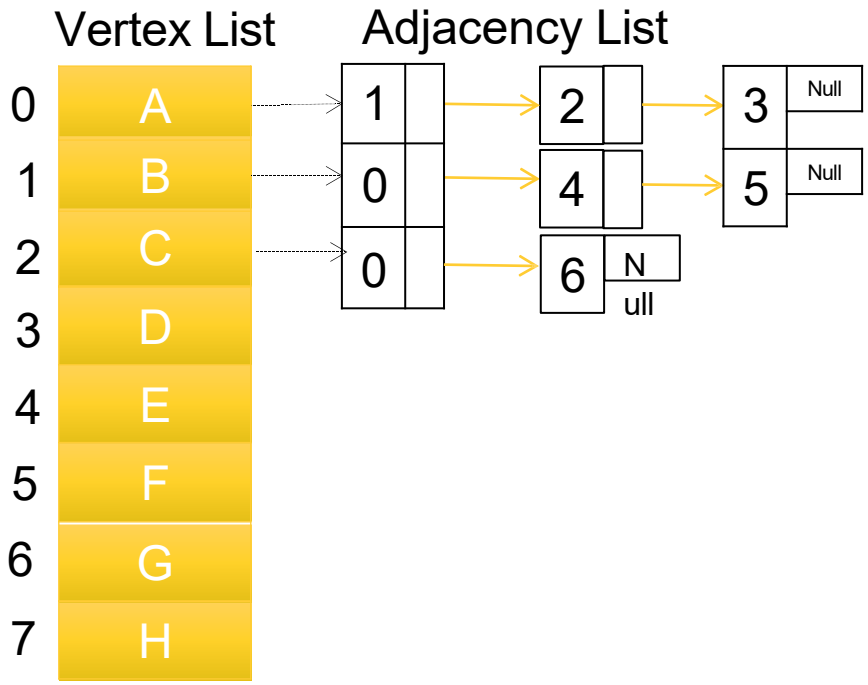


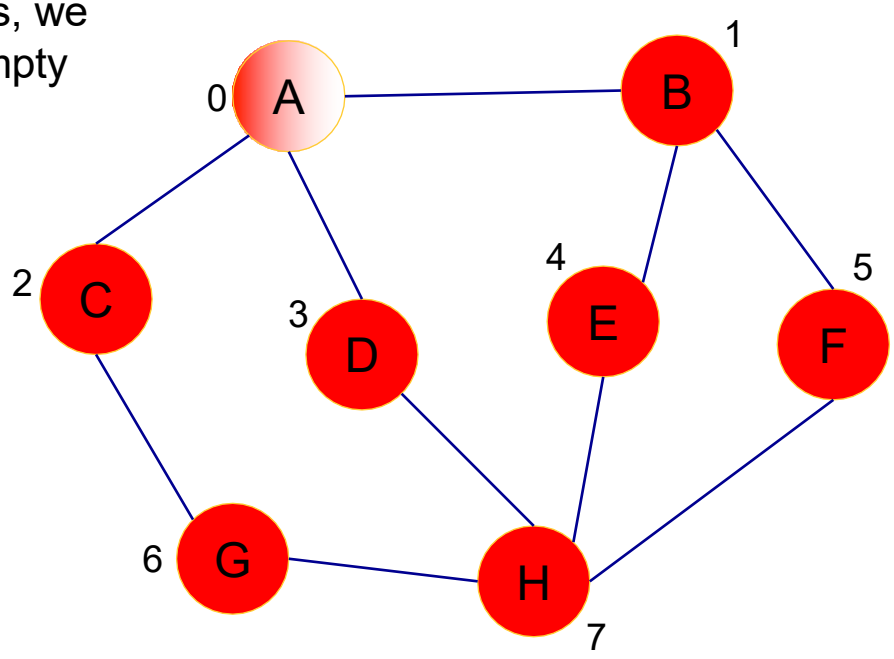


stack: A

Depth-First Search (DFS)

Repeat the procedure for A:
Since A has no un-visited adjacent vertices, we pop A off of the stack. The stack is now empty Indicating that the traversal is complete.



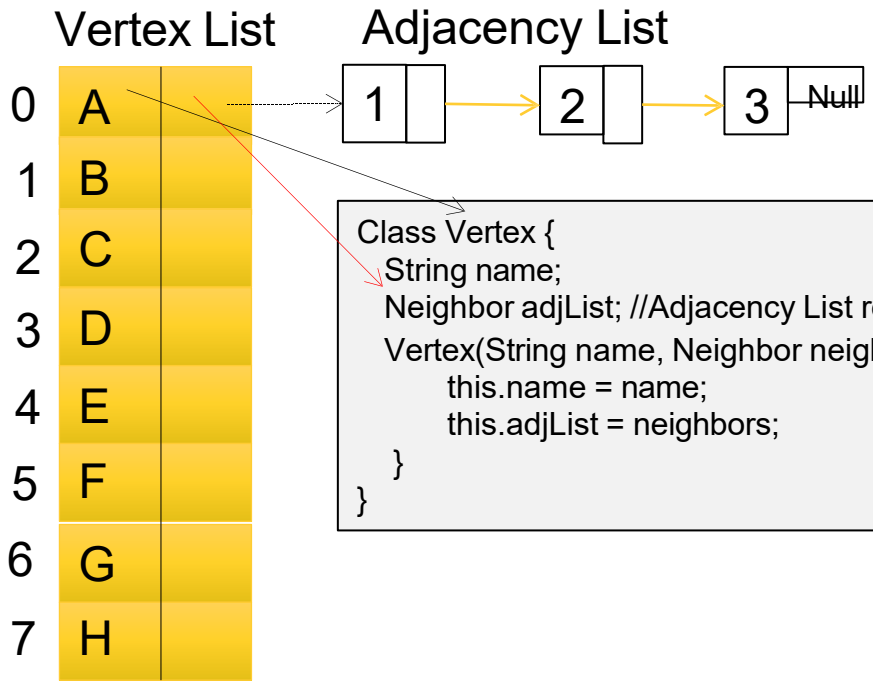


stack: empty

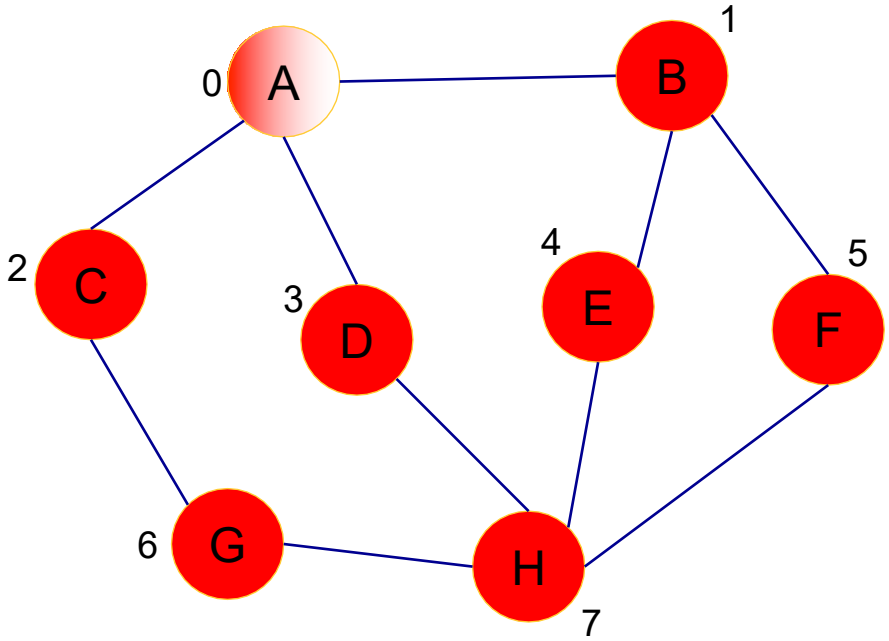
Depth-First Search (DFS)

Each element of the Vertex List array stores an object of class Vertex and a reference to a Neighbor;

Neighbor objects are the nodes within the adjacency list.

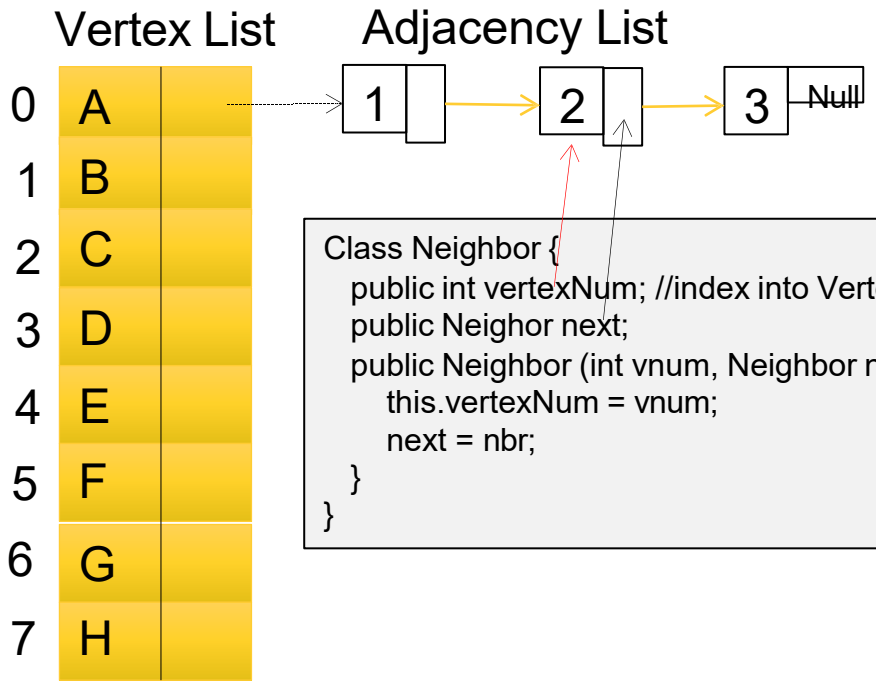



```
Class Vertex {
   String name;
   Neighbor adjList; //Adjacency List reference
   Vertex(String name, Neighbor neighbors) {
         this.name = name;
         this.adjList = neighbors;
   }
}
```

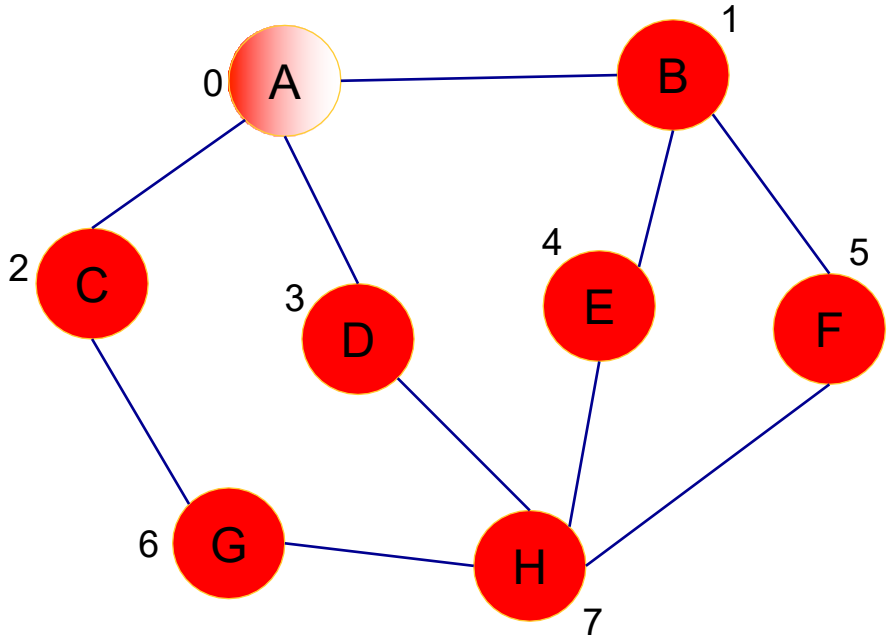


Depth-First Search (DFS)

Neighbor objects are the nodes in the adjacency list.

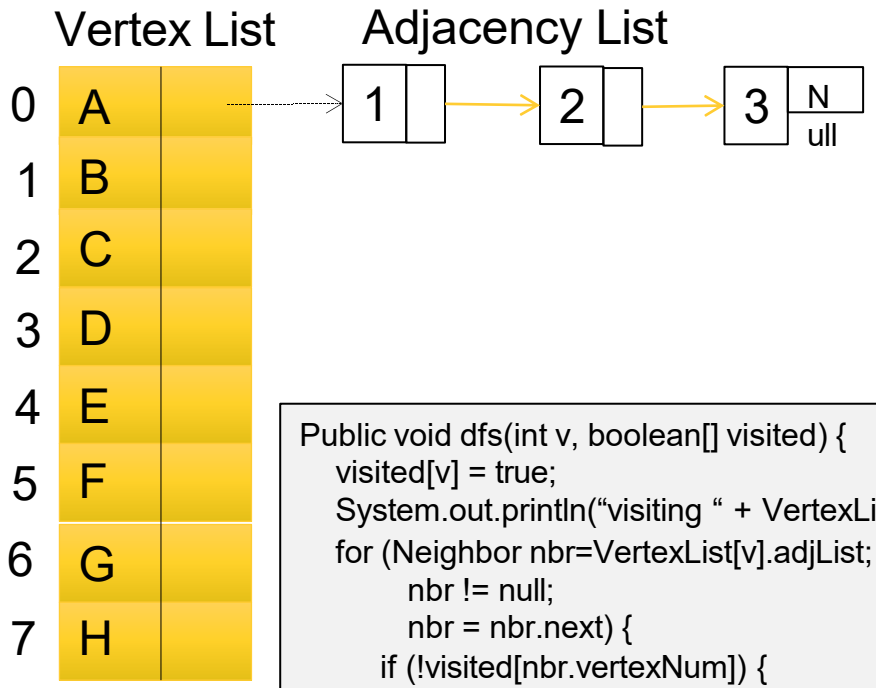


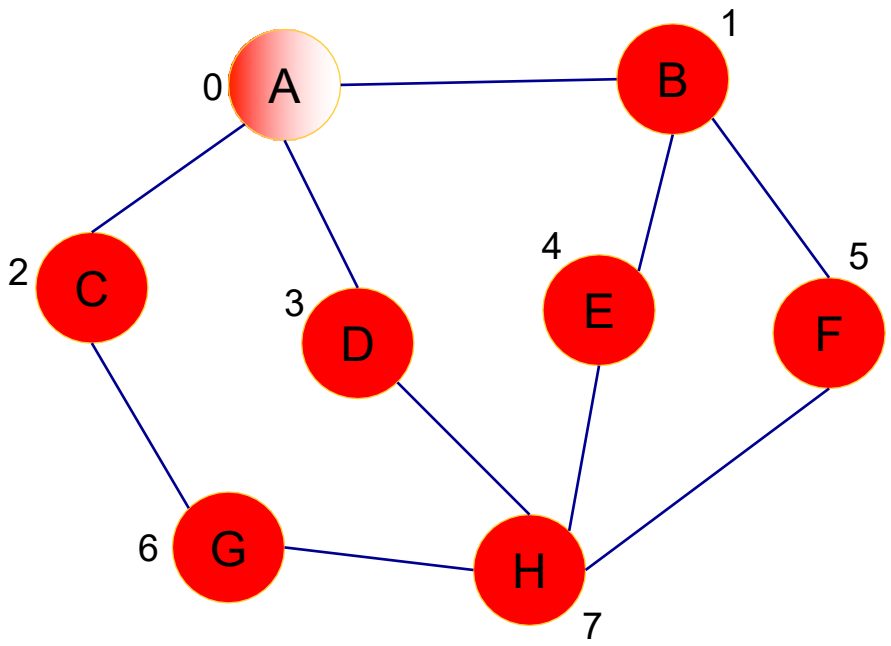

```
Class Neighbor {
   public int vertexNum; //index into Vertex List
   public Neighor next;
   public Neighbor (int vnum, Neighbor nbr) {
       this.vertexNum = vnum;
       next = nbr;
   }
}
```




```
Class Vertex {
   String name;
   Neighbor adjList; //Adjacency List reference
   Vertex(String name, Neighbor neighbors) {
         this.name = name;
         this.adjList = neighbors;
    }
}
```

Depth-First Search (DFS)



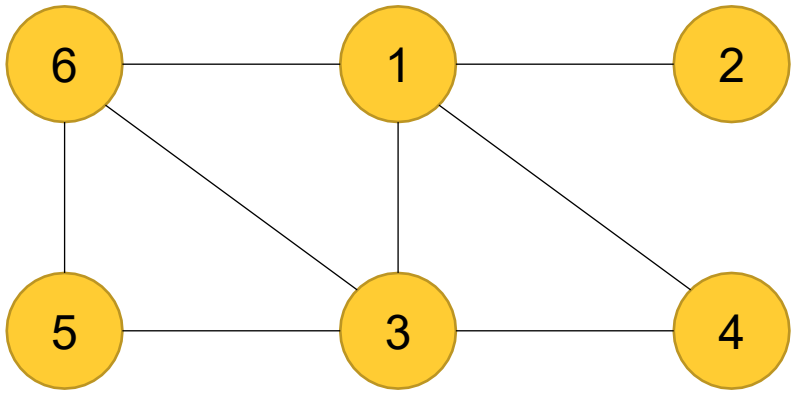



```
Class Vertex {
   String name;
   Neighbor adjList; //Adjacency List reference
   Vertex(String name, Neighbor neighbors) {
         this.name = name;
         this.adjList = neighbors;
    }
}
```

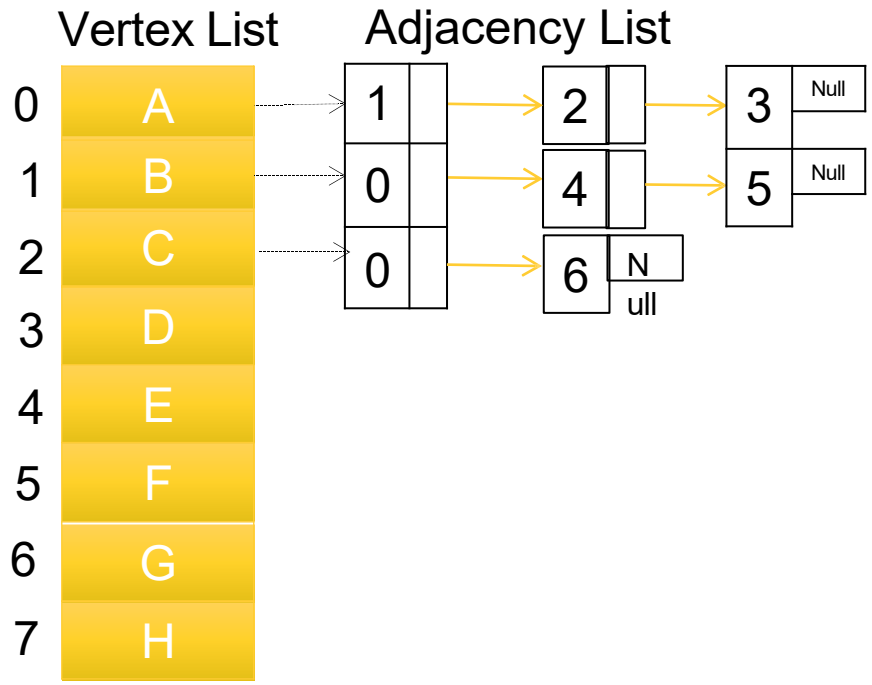
```
Class Neighbor {
   public int vertexNum; //index into Vertex List
   public Neighor next;
   public Neighbor (int vnum, Neighbor nbr) {
      this.vertexNum = vnum;
      next = nbr;
   }
}
```

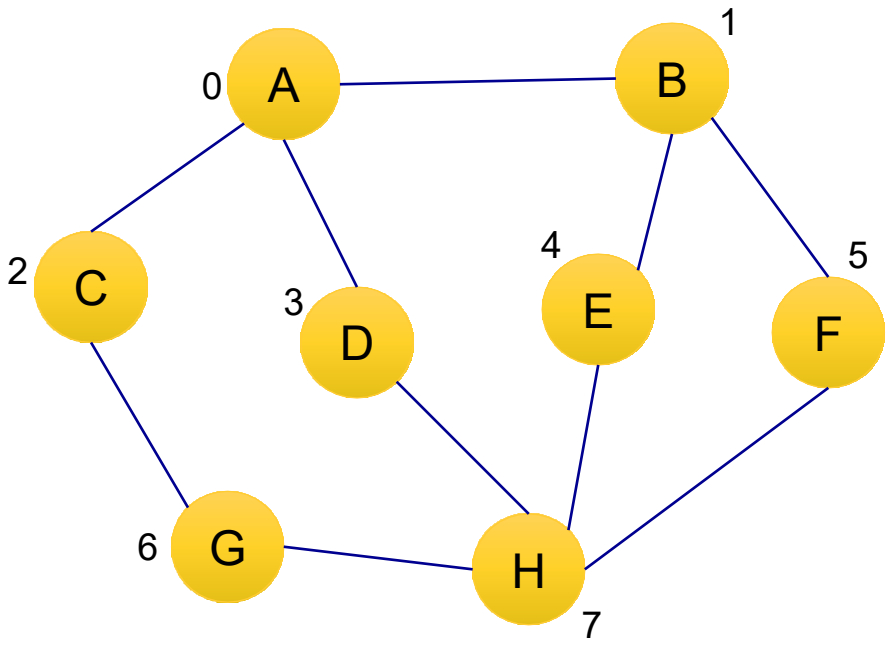
```
Public void dfs(int v, boolean[] visited) {
    visited[v] = true;
    System.out.println("visiting " + VertexList[v].name);
    for (Neighbor nbr=VertexList[v].adjList;
            nbr != null;
            nbr = nbr.next) {
        if (!visited[nbr.vertexNum]) {
            System.out.println( VertexList[v].name "—" +
                    VertexList[nbr.vertexNum].name);
            dfs(nbr.vertexNum, visited);
        }
    }
}
```

In-class: DFS Graph



Breadth-First Search (BFS)

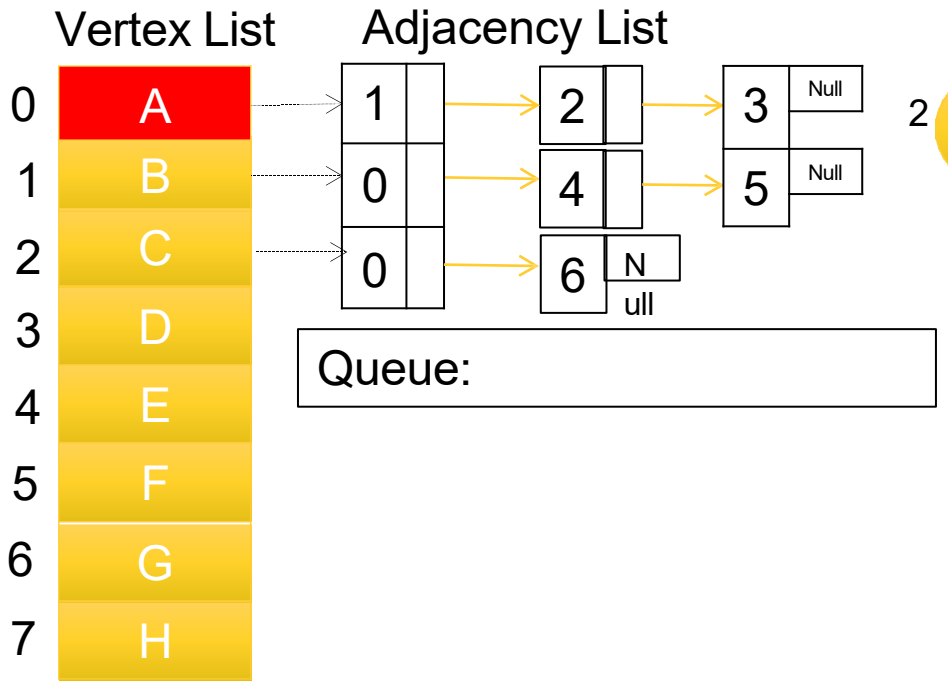


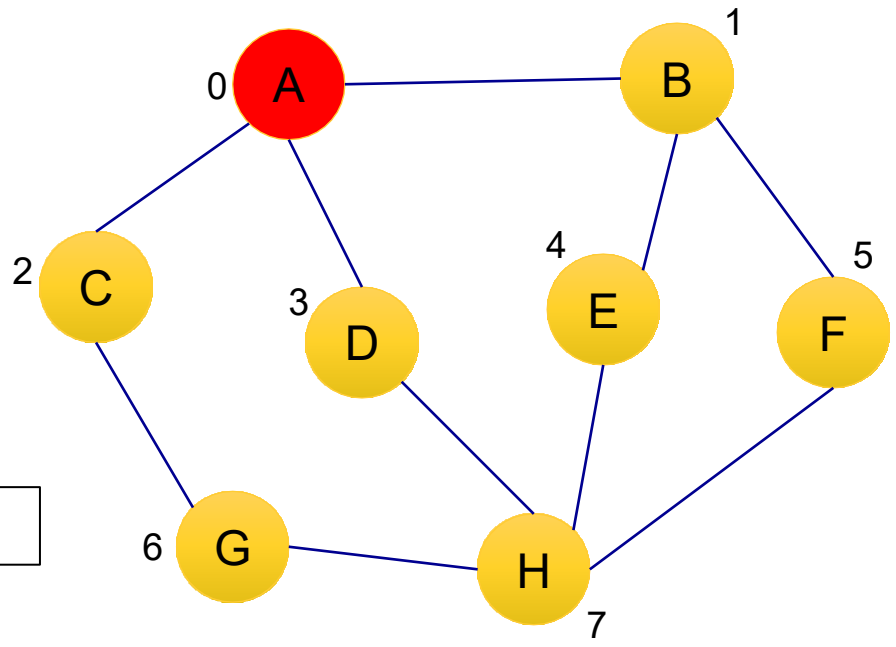


For BFS we use a **queue**

Breadth-First Search (BFS)

Whereas for DFS we use a stack as our supporting data structure, for BFS we use a **queue.**

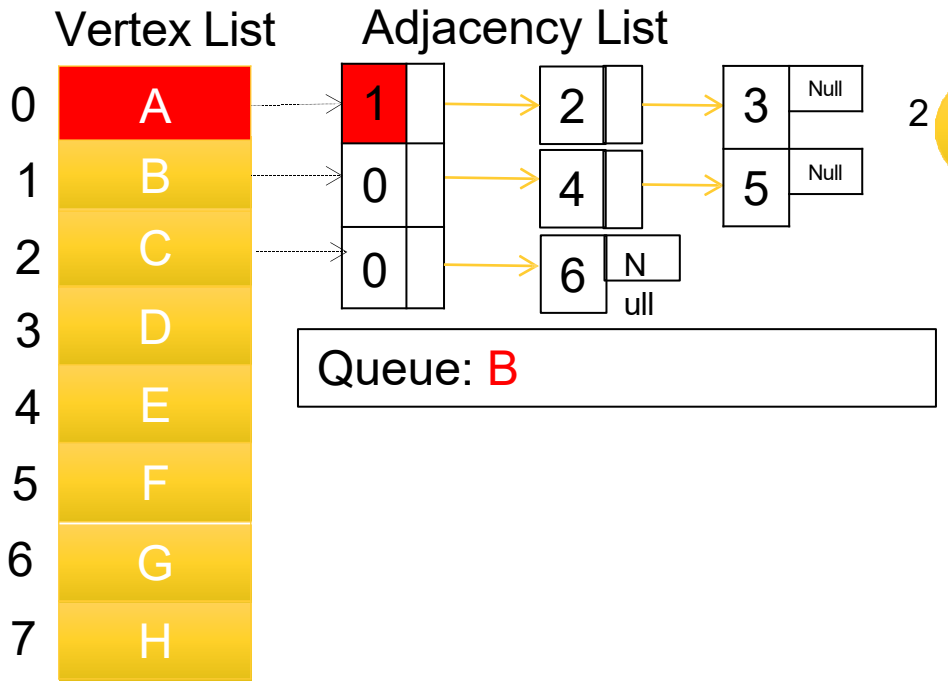




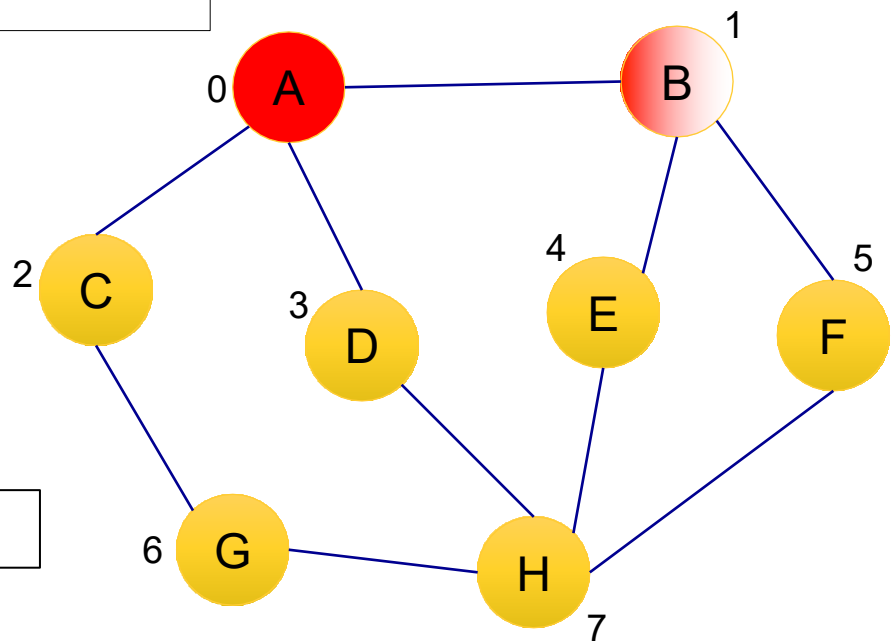
Visited: A

Breadth-First Search (BFS)

Using the adjacency list, we visit A's adjacent neighbors in order, the first being vertex B, We enqueue B. Note: We don't **move** to B, We're still **working** on vertex A.



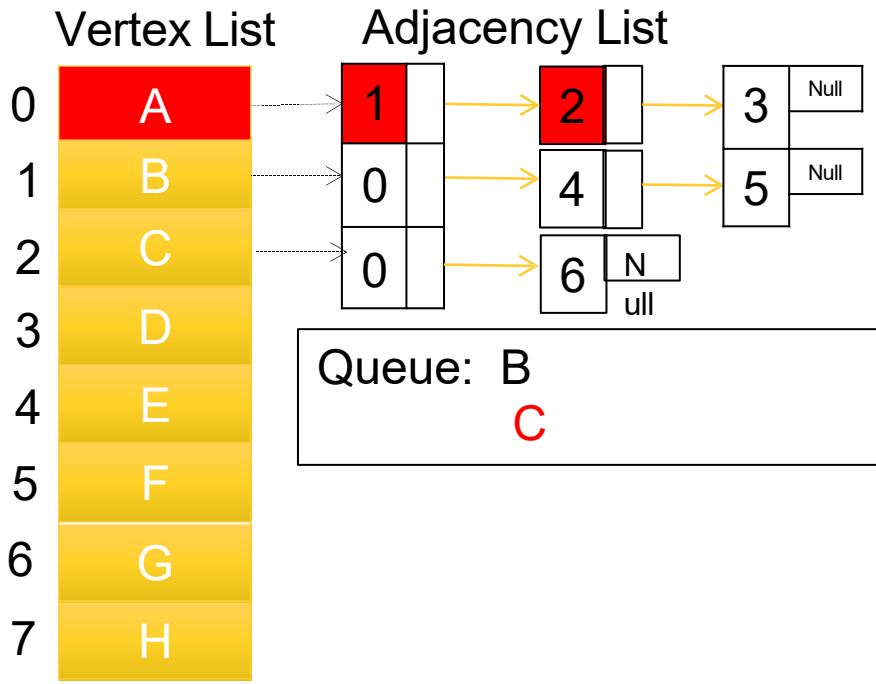
Queue: B



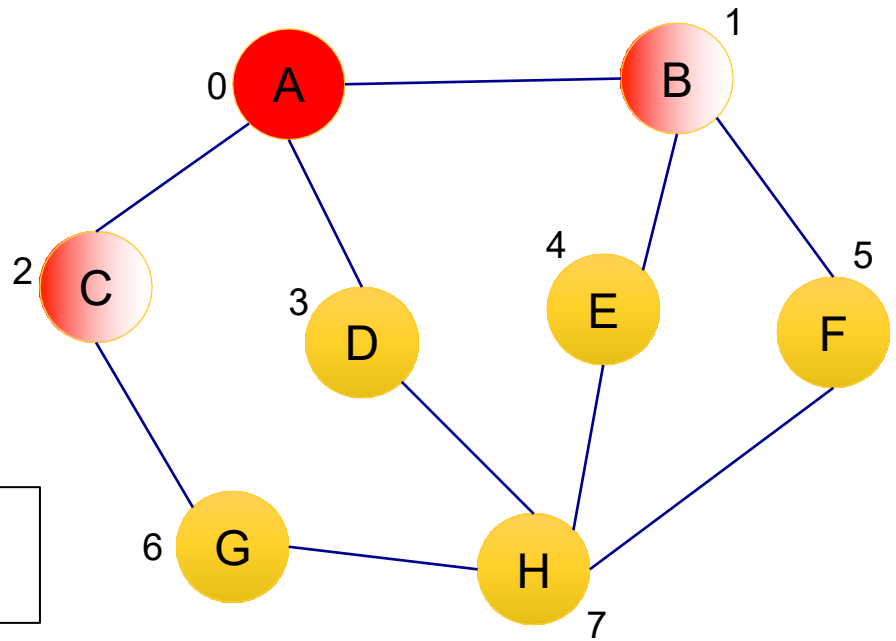
Visited: A B

Breadth-First Search (BFS)

Since we're still working on vertex A, we now **visit** the next vertex in the adjacency list **C**, and we enqueue C.



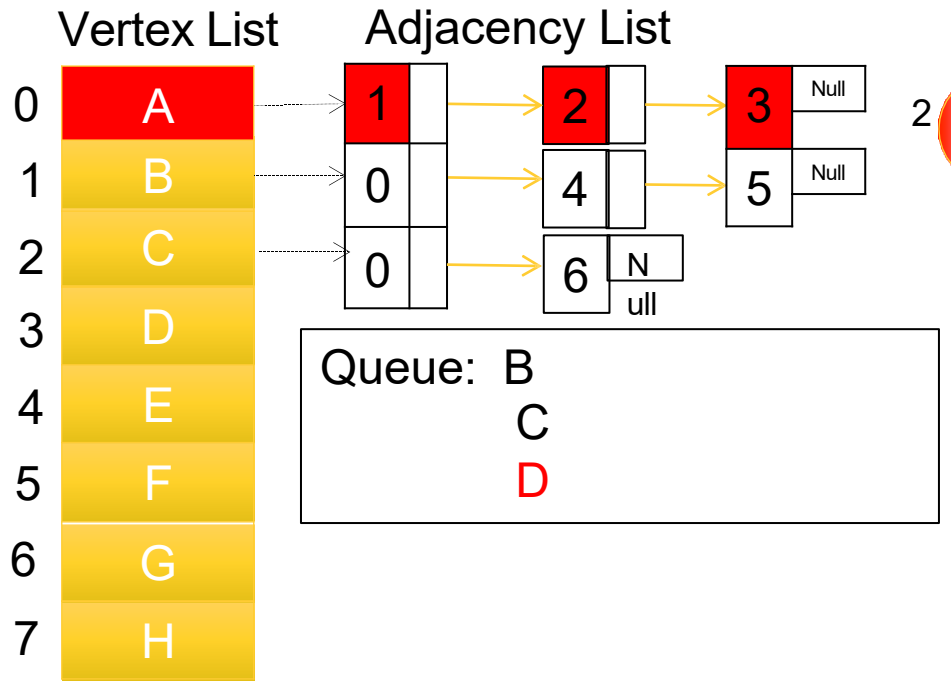
Queue: B
C



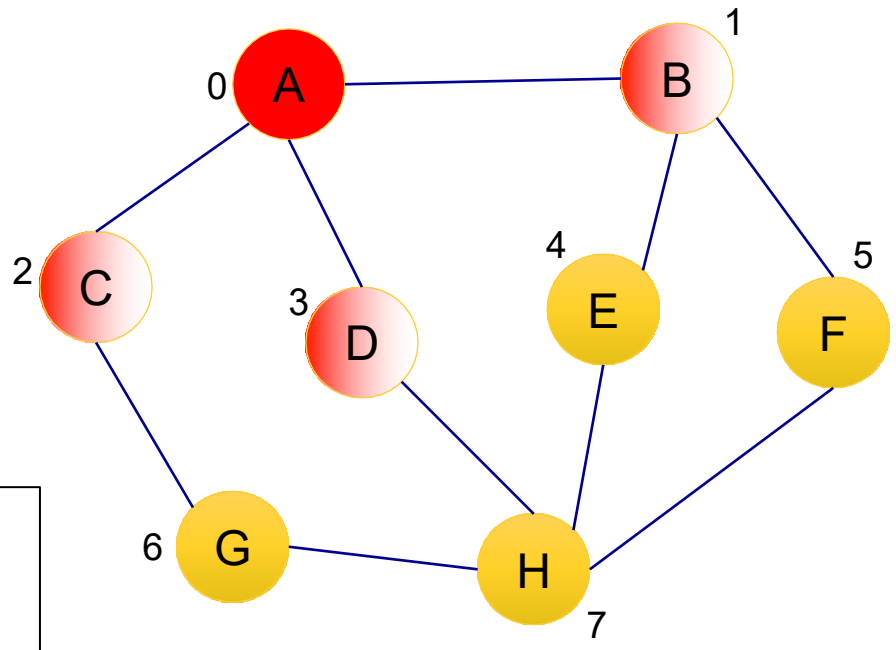
Visited: A B C

Breadth-First Search (BFS)

Repeat the process and visit A's one remaining adjacent vertex: D We enqueue D



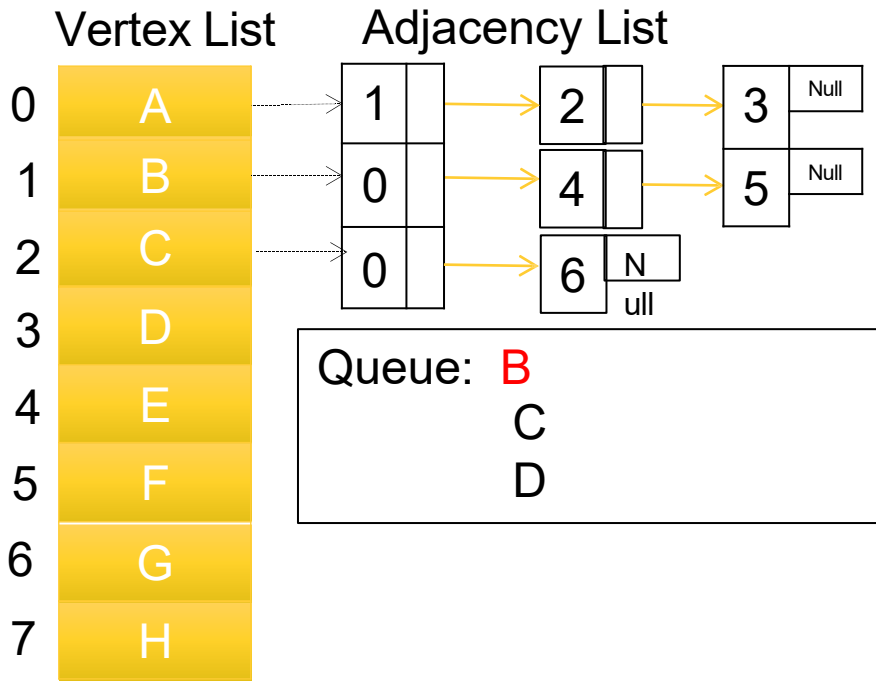
Queue: B
C
D



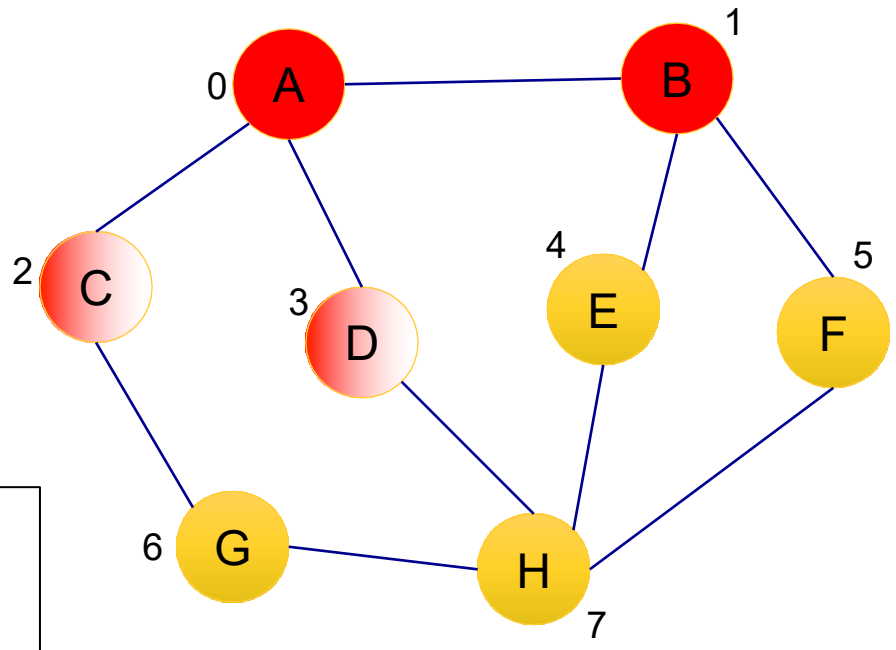
Visited: A B C D

Breadth-First Search (BFS)

Since A has no other adjacent vertices, we're finished with A. We will now work on the vertex at the head of the queue: B



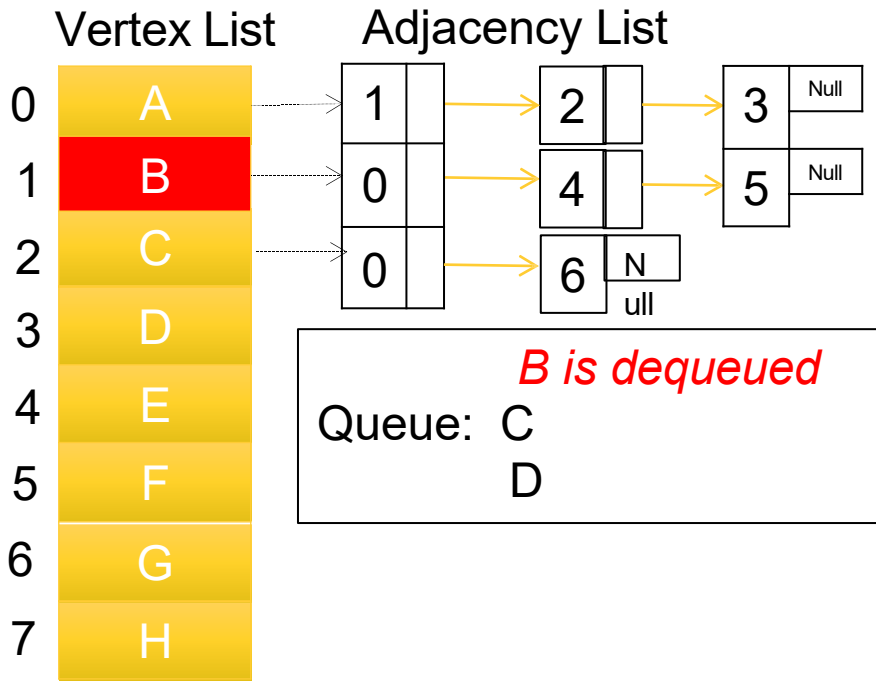
Queue: B
C
D



Visited: A B C D

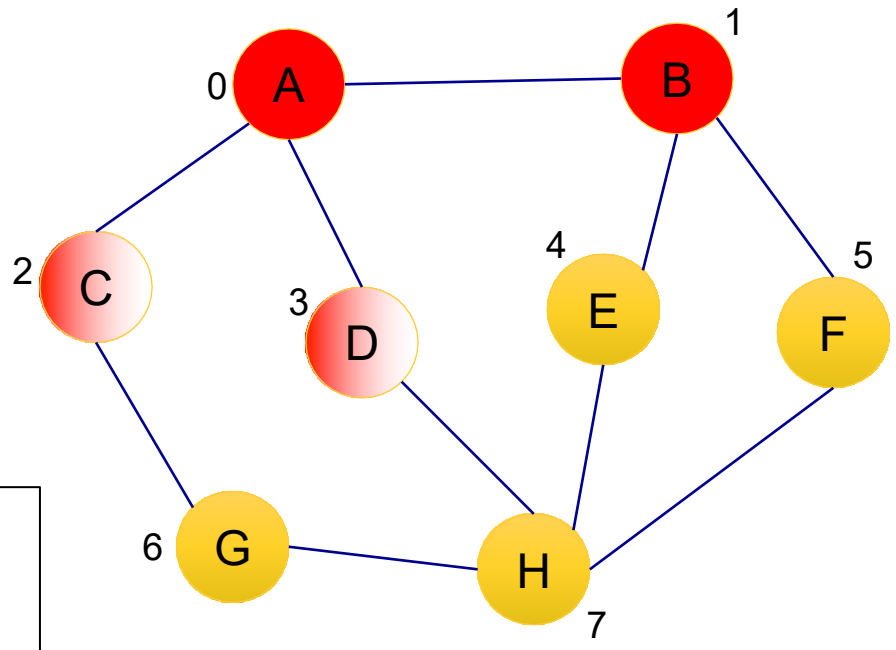
Breadth-First Search (BFS)

Since we are now working on B, we dequeue B



B is dequeued

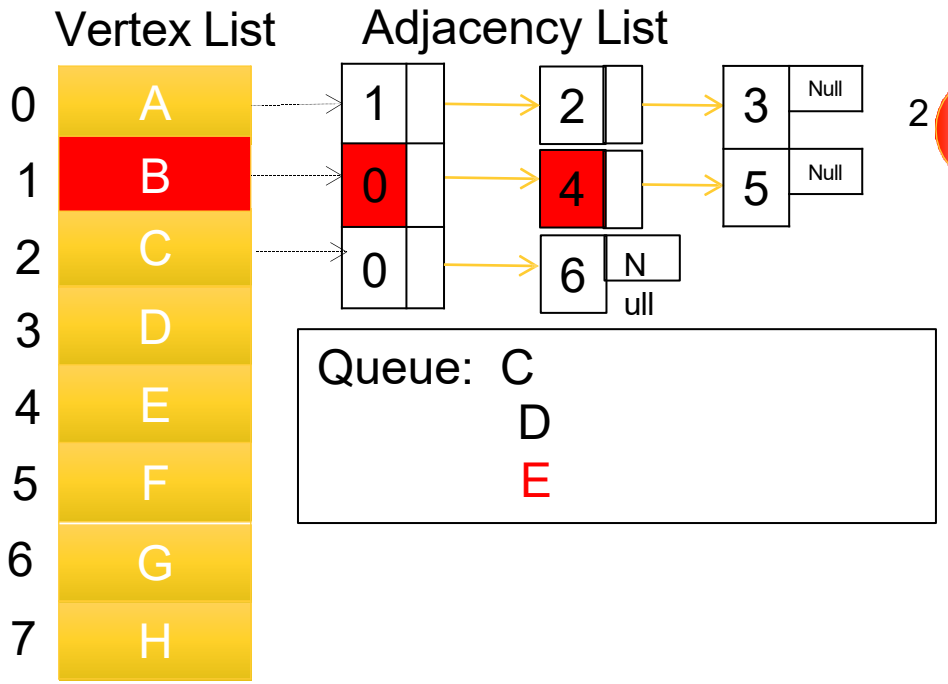
Queue: C
D



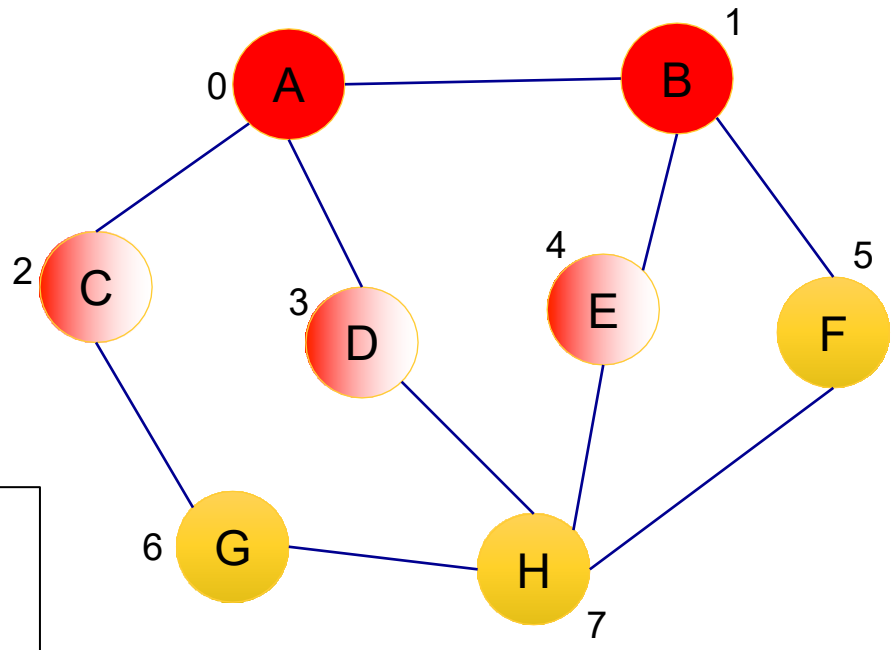
Visited: A B C D

Breadth-First Search (BFS)

We now use the adjacency list to visit all of the vertices adjacent to B that have not already been visited. We enqueue E.



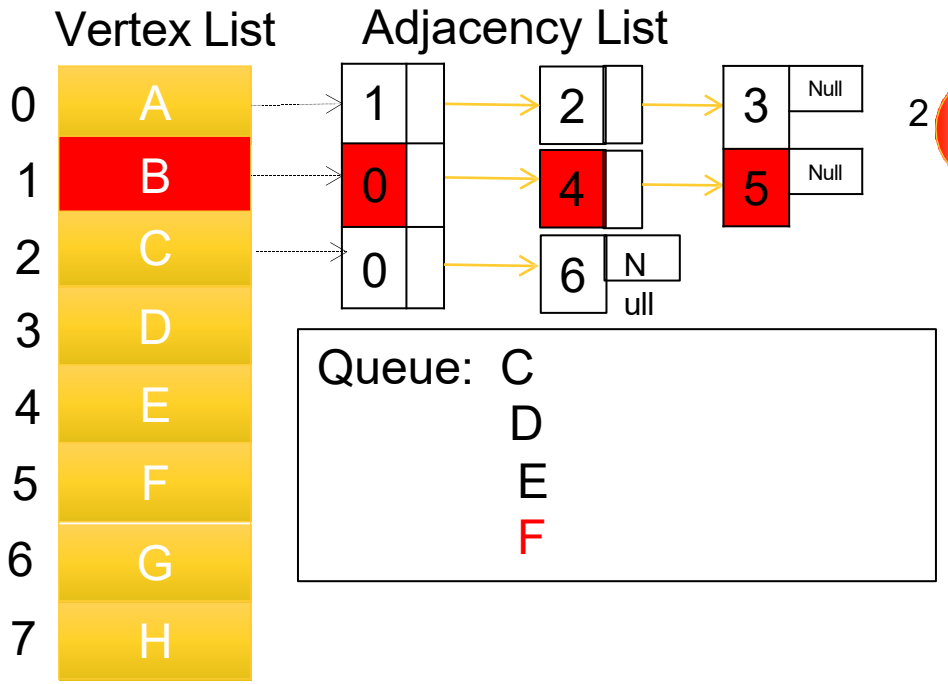
Queue: C
D
E



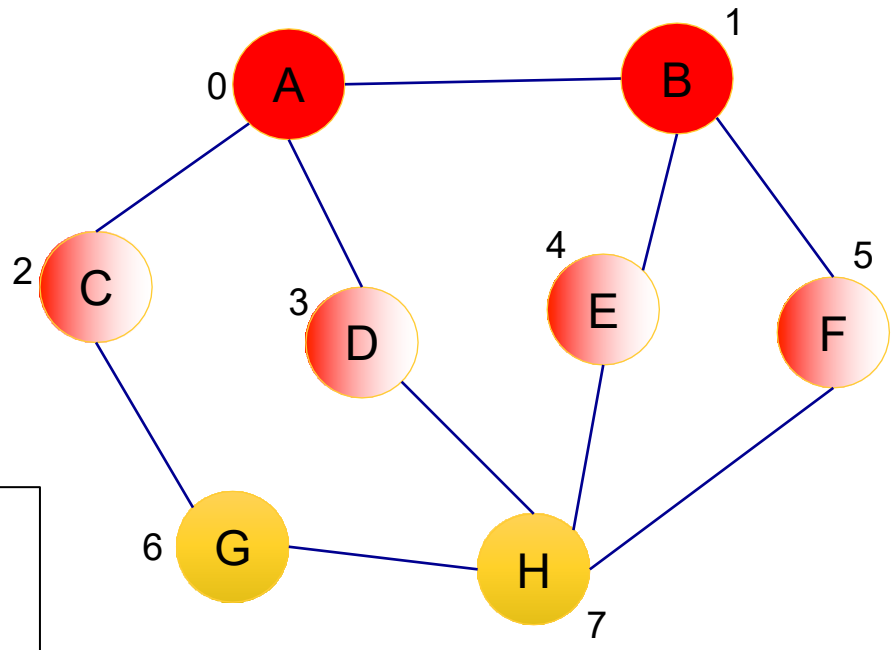
Visited: A B C D E

Breadth-First Search (BFS)

Repeat the process for B's one remaining adjacent vertex: F



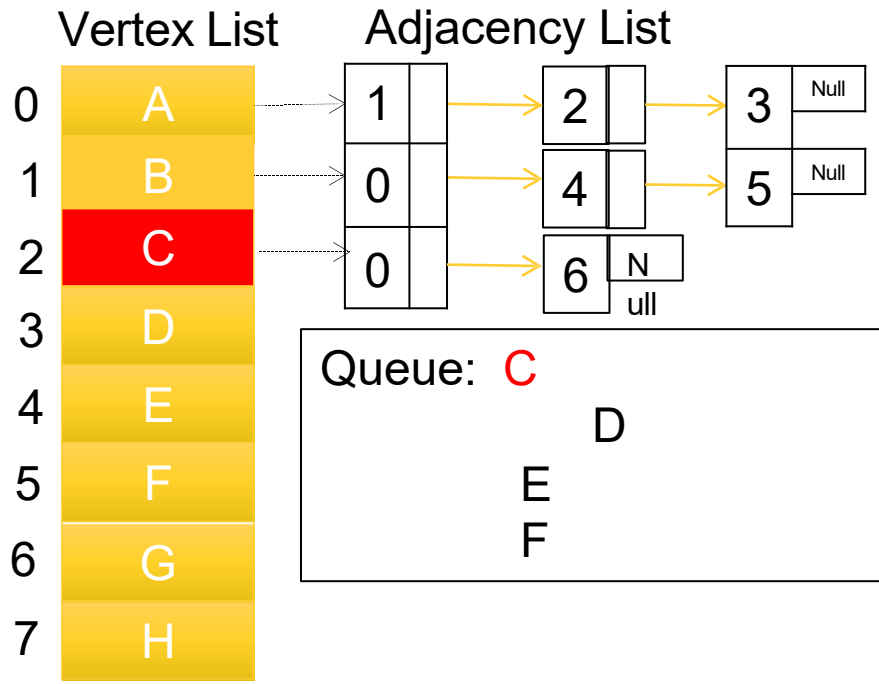
Queue: C
D
E
F



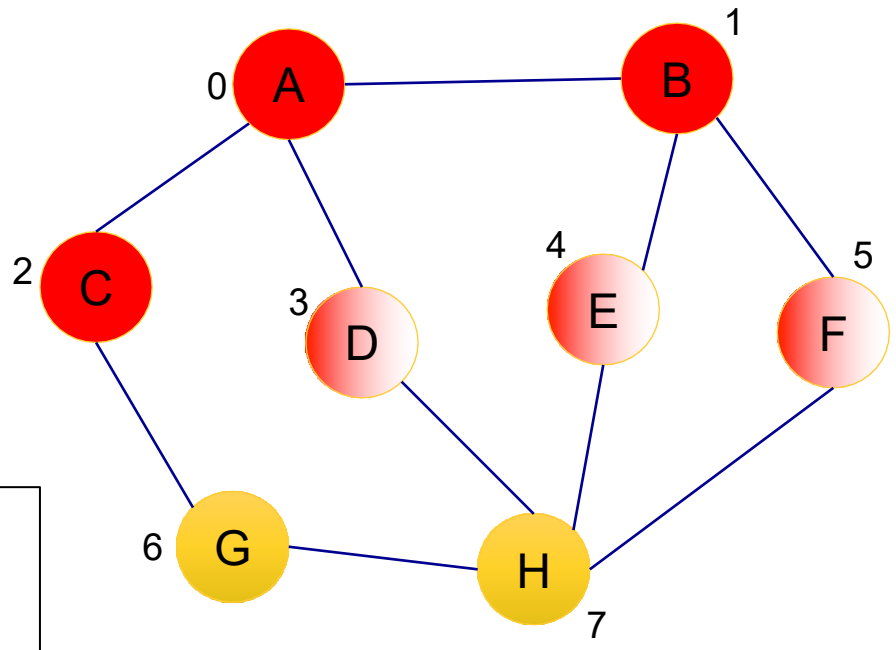
Visited: A B C D E F

Breadth-First Search (BFS)

B has no other adjacent vertices, so we move on to the next vertex on the queue C.



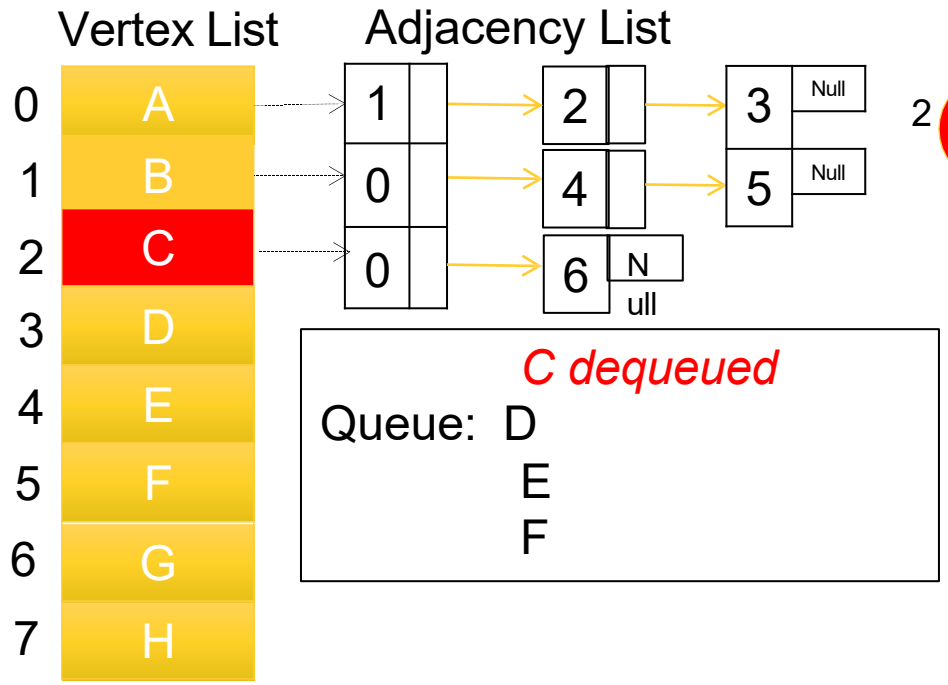
Queue: C
D
E
F



Visited: A B C D E F

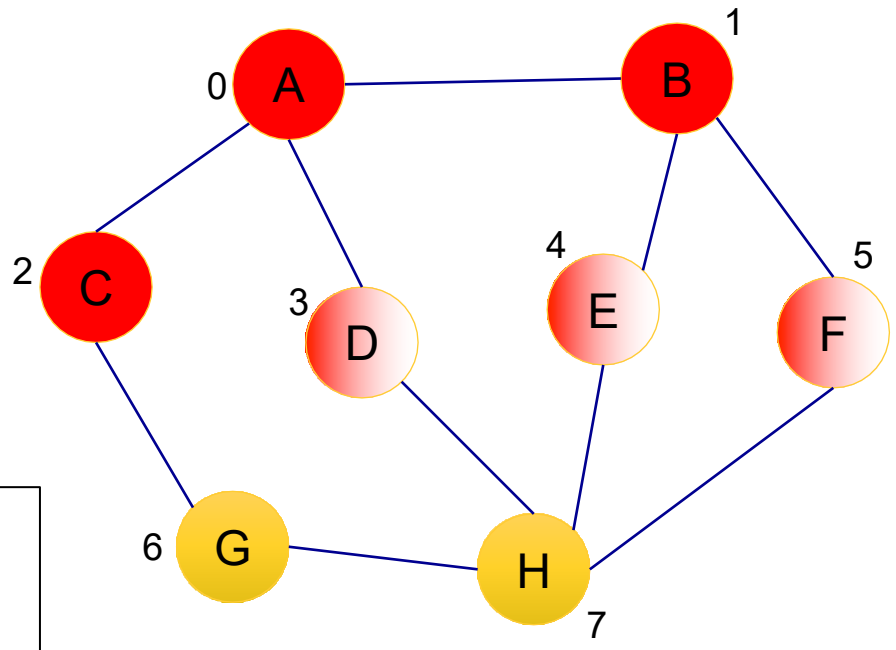
Breadth-First Search (BFS)

We dequeue C.



C dequeued

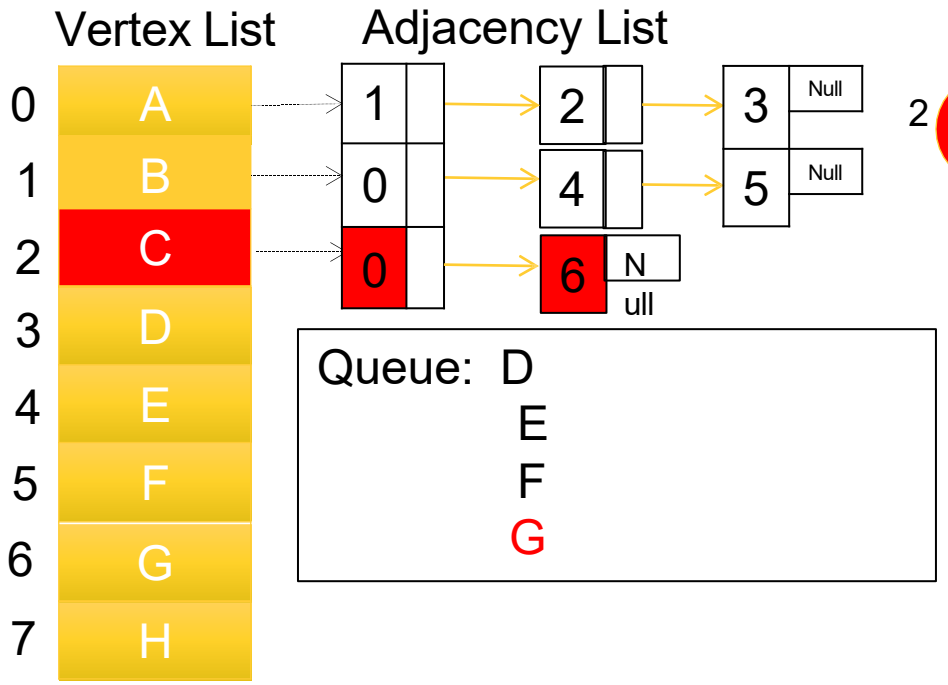
Queue: D
E
F



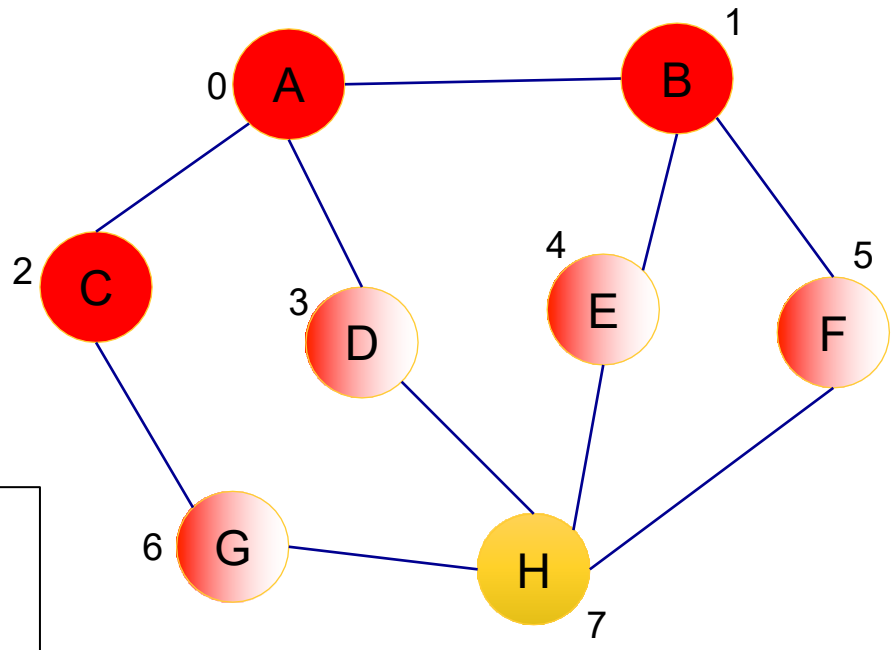
Visited: A B C D E F

Breadth-First Search (BFS)

C's only previously unvisited adjacent vertex is G, which we enqueue.



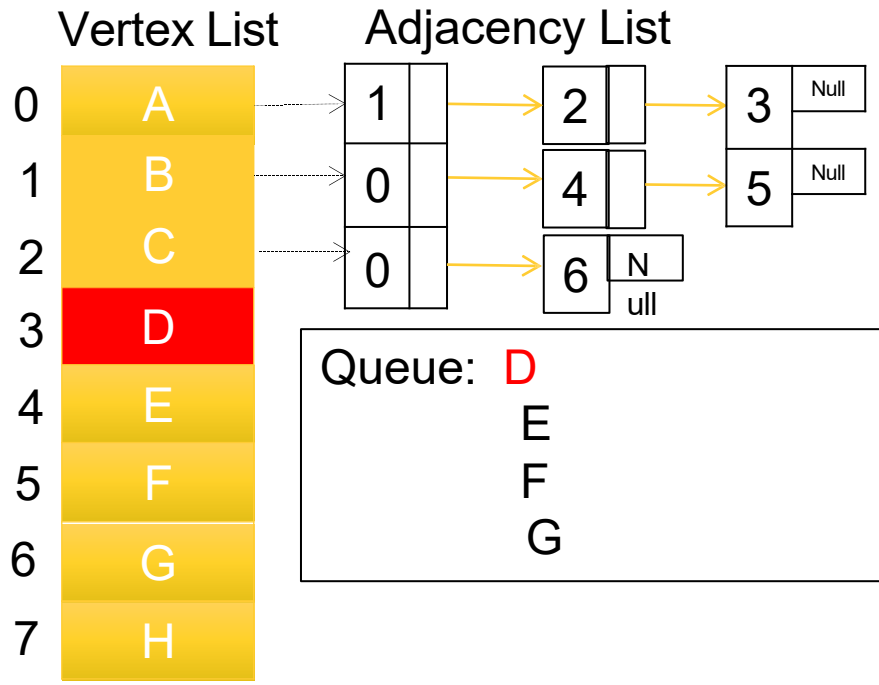
Queue: D
E
F
G



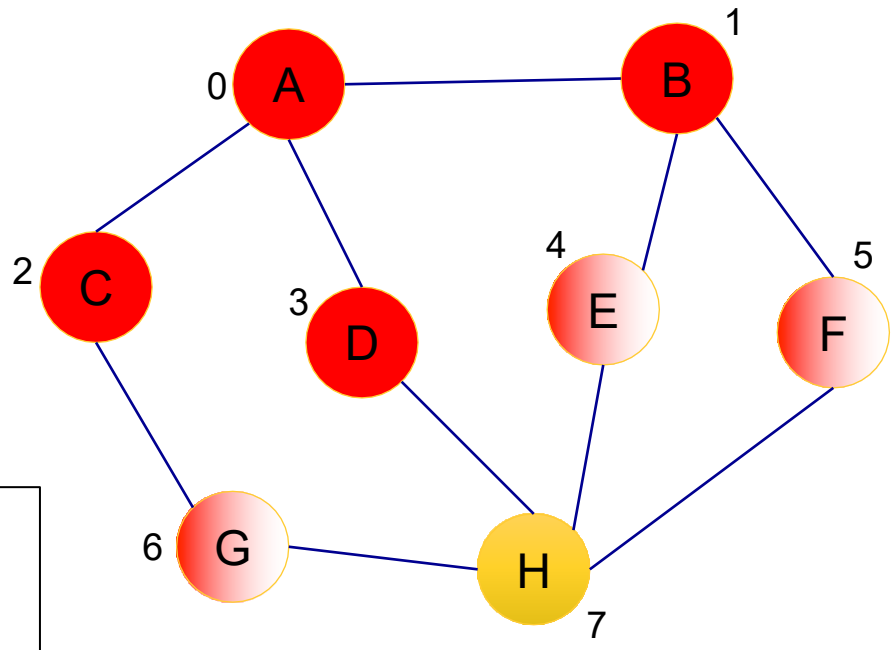
Visited: A B C D E F G

Breadth-First Search (BFS)

Our work with C is complete, so we move on to the next vertex on the Queue: D



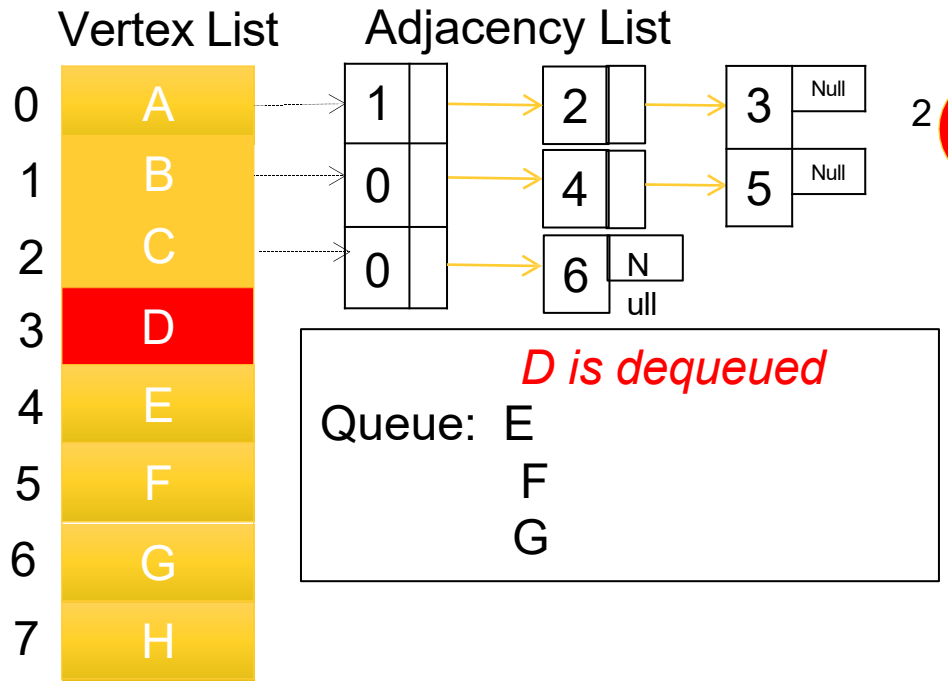
Queue: D
E
F
G



Visited: A B C D E F G

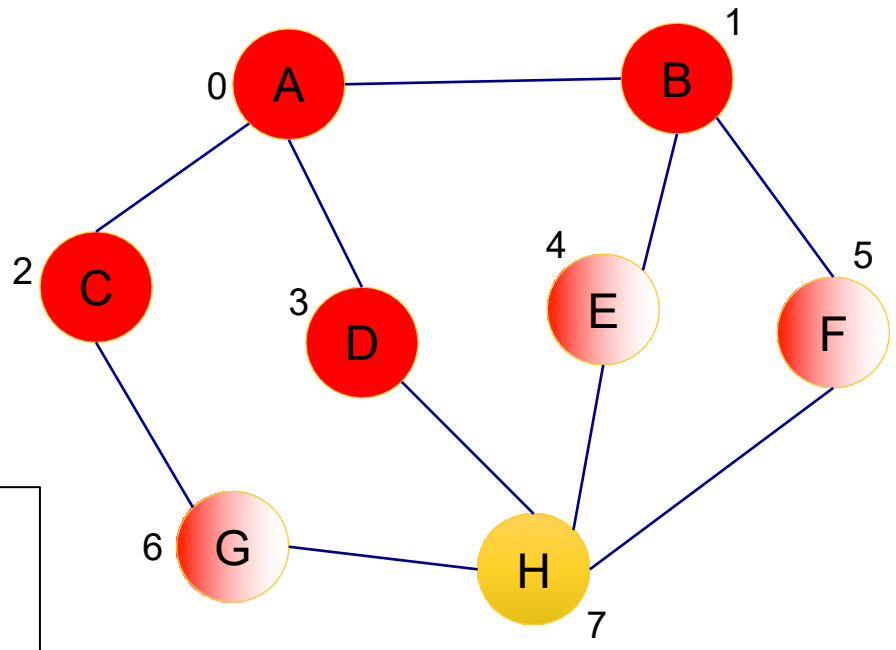
Breadth-First Search (BFS)

We dequeue: D



D is dequeued

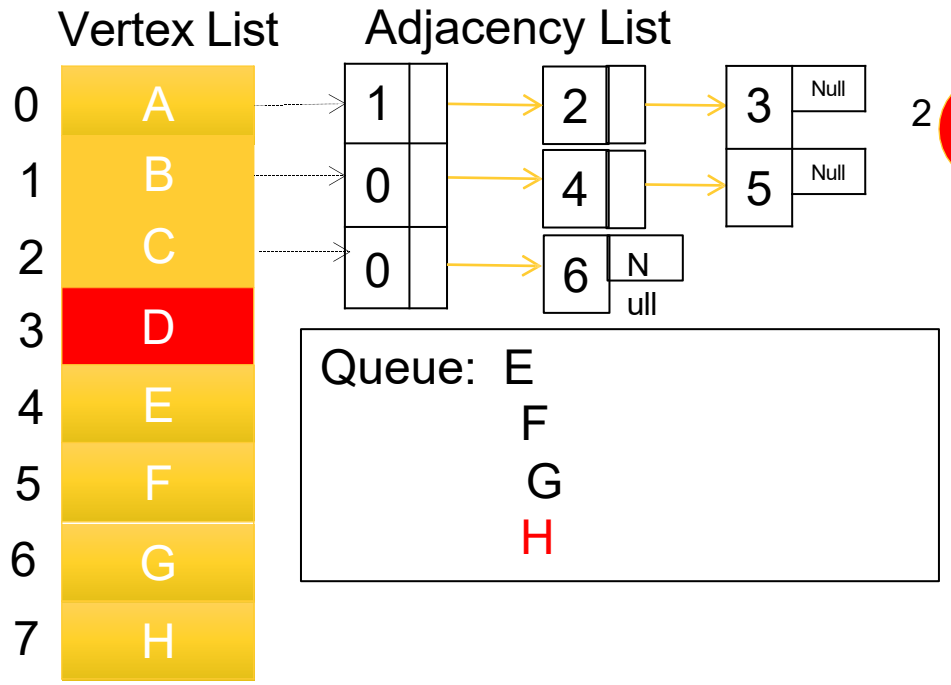
Queue: E
F
G



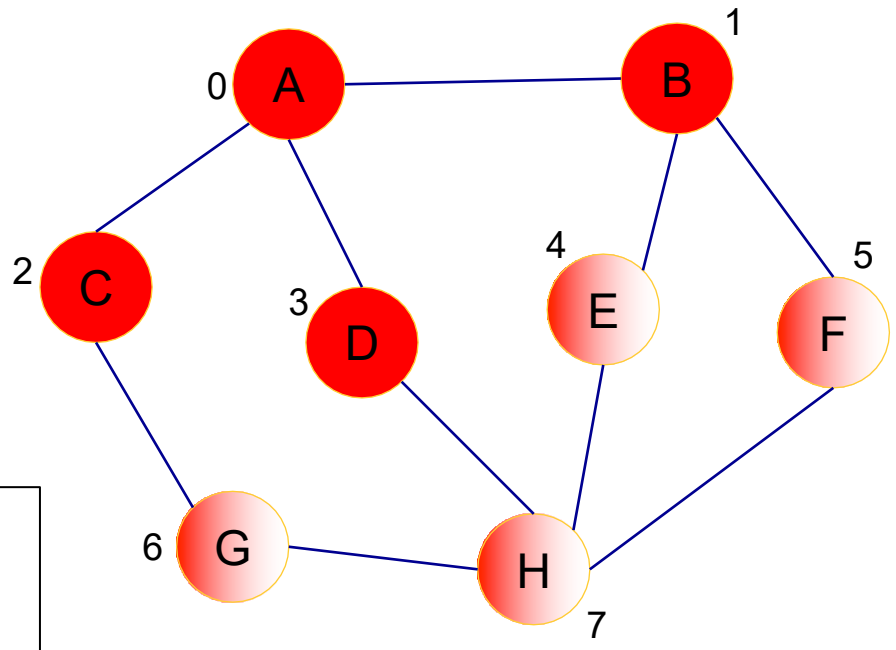
Visited: A B C D E F G

Breadth-First Search (BFS)

We visit D's only unvisited adjacent neighbor: H



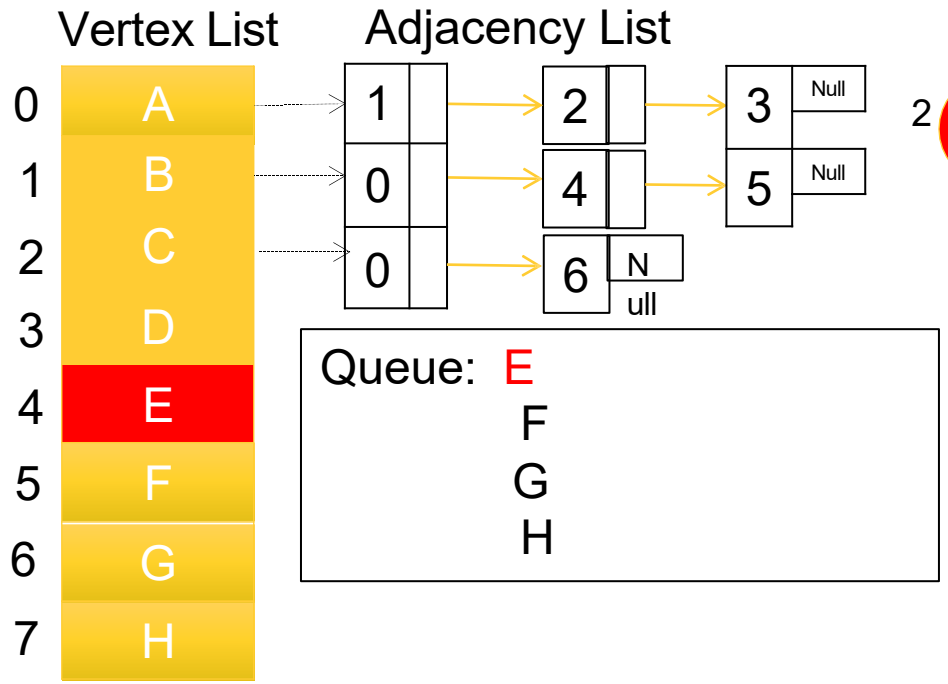
Queue: E
F
G
H



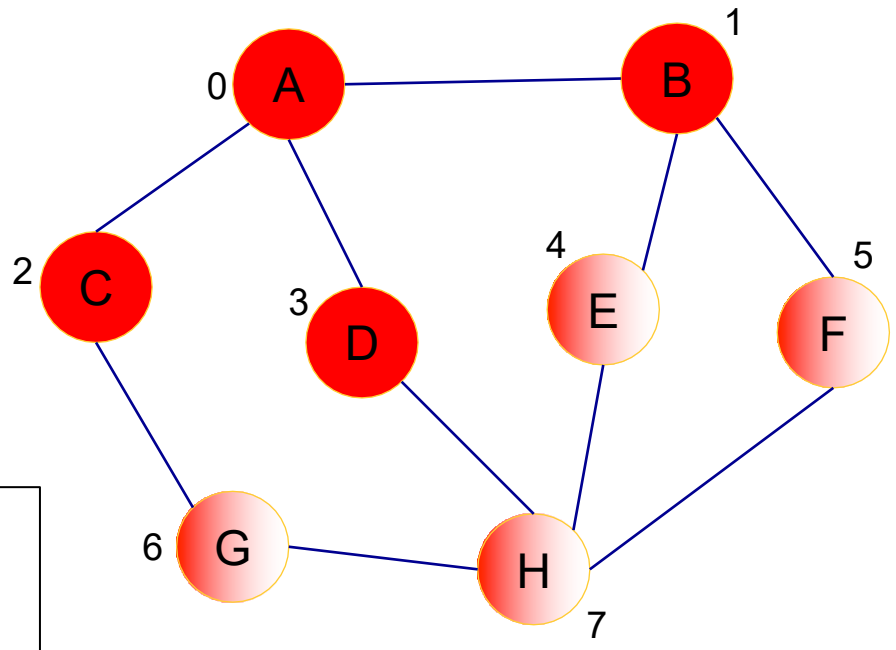
Visited: A B C D E F G H

Breadth-First Search (BFS)

D has no other previously unvisited vertices, so we move on to the next vertex in the queue E.



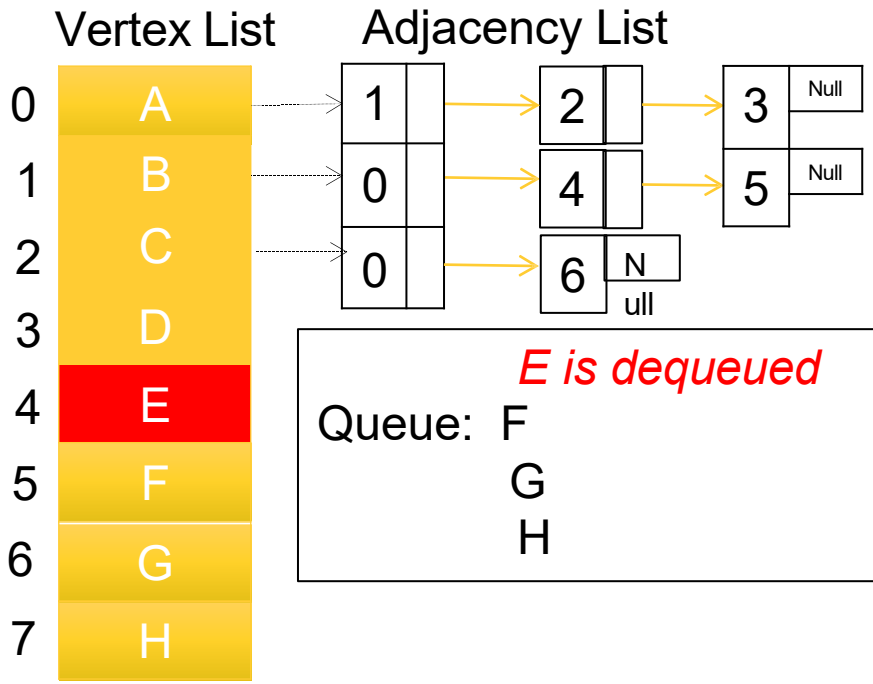
Queue: E
F
G
H



Visited: A B C D E F G H

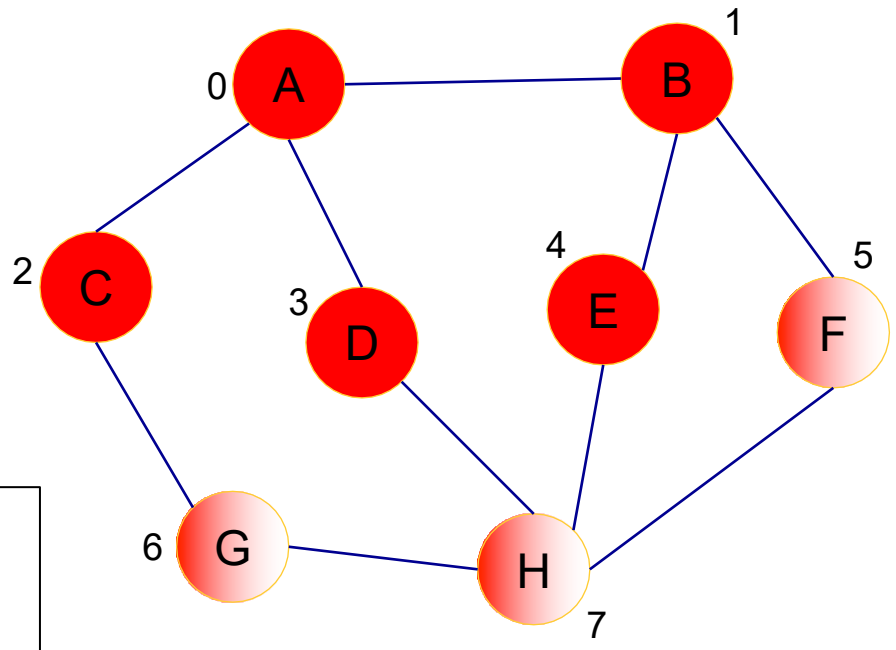
Breadth-First Search (BFS)

D has no other previously unvisited vertices, so we move on to the next vertex in the queue E.



E is dequeued

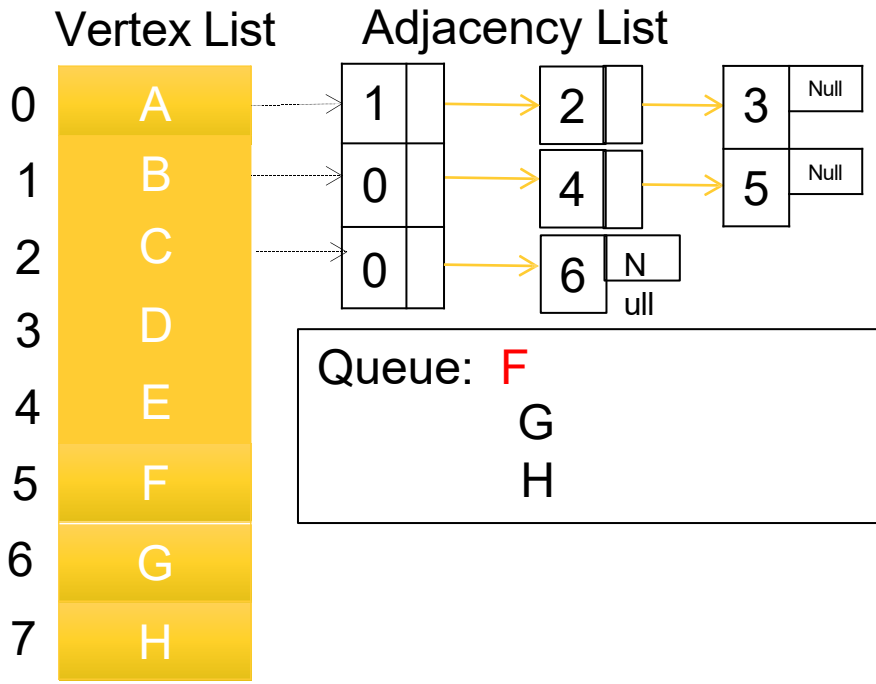
Queue: F
G
H



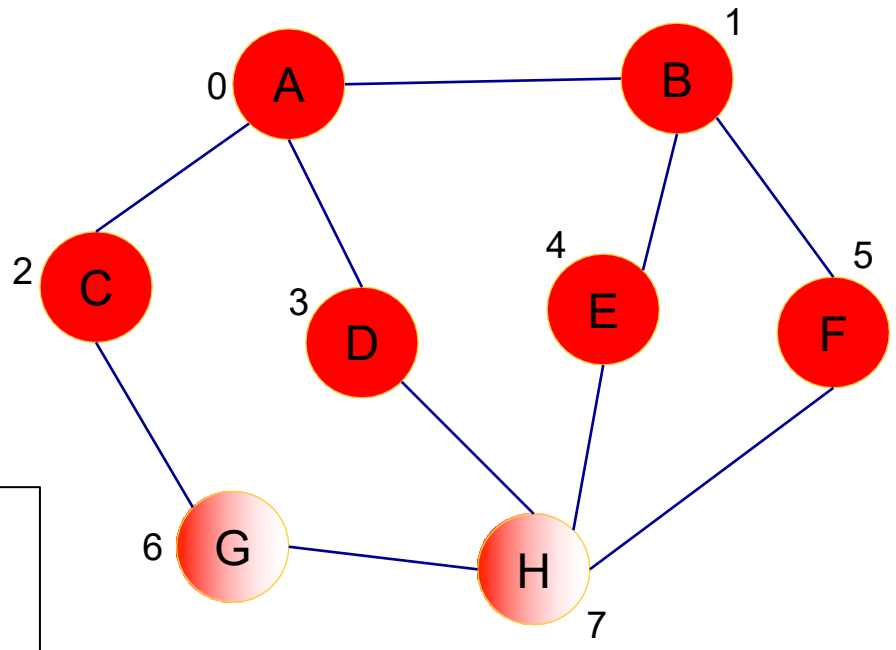
Visited: A B C D E F G H

Breadth-First Search (BFS)

E has no previously unvisited vertices, so we move on to the next vertex in the queue F.



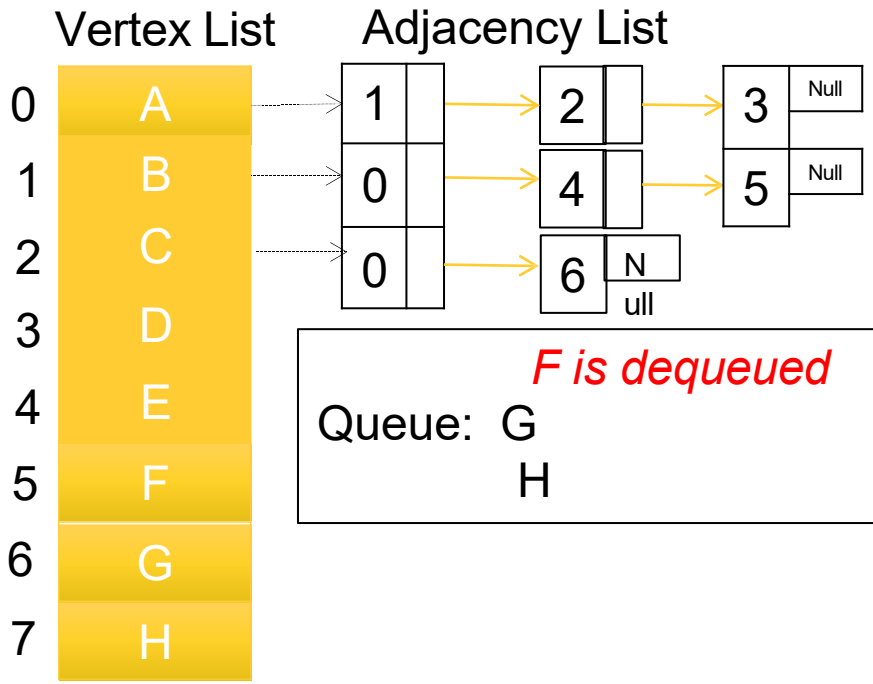
Queue: F
G
H



Visited: A B C D E F G H

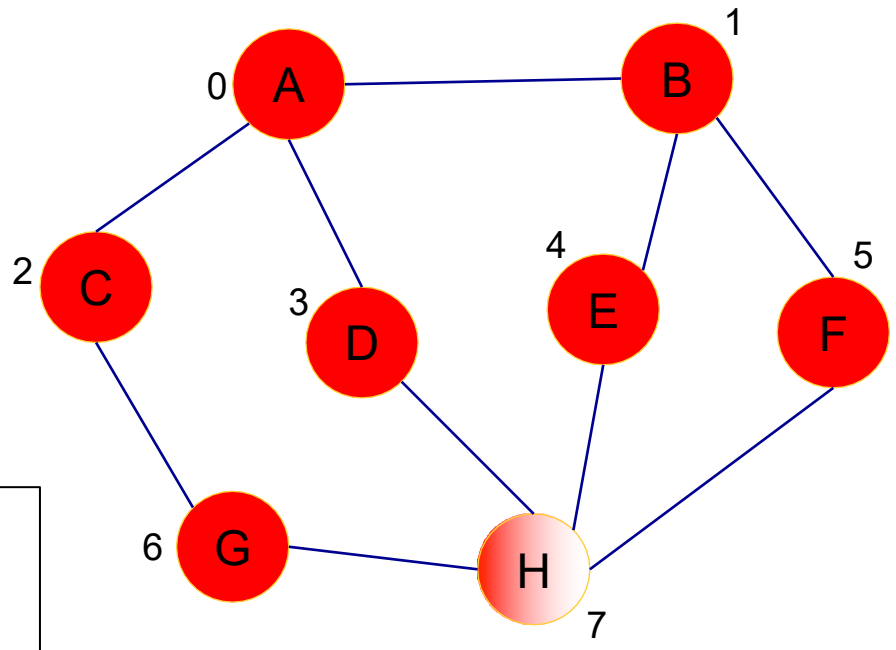
Breadth-First Search (BFS)

F has no previously unvisited vertices, so we move on to the next vertex in the queue G.



F is dequeued

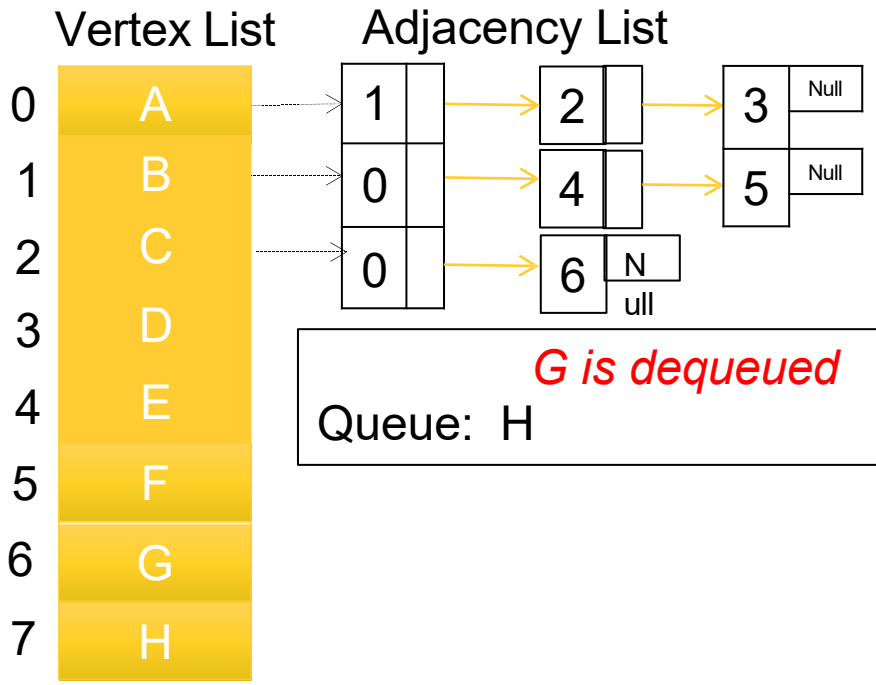
Queue: G
H



Visited: A B C D E F G H

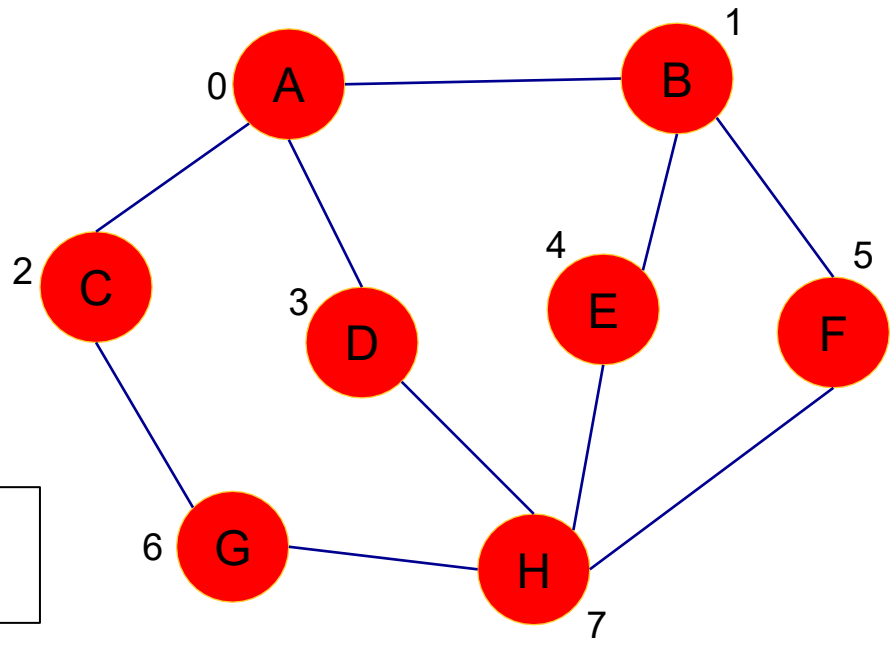
Breadth-First Search (BFS)

G has no previously unvisited vertices, so we move on to the next vertex in the queue H.



G is dequeued

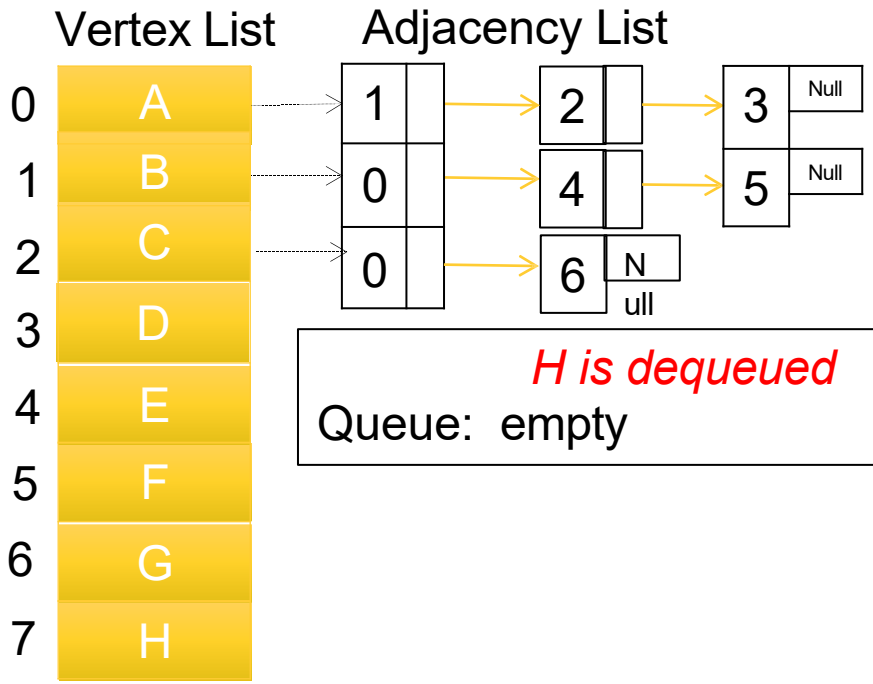
Queue: H



Visited: A B C D E F G H

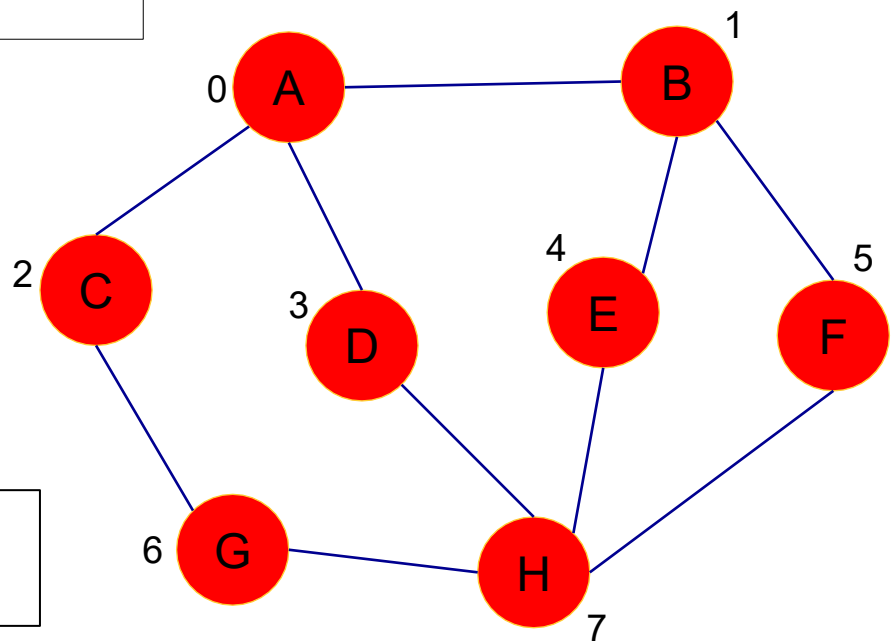
Breadth-First Search (BFS)

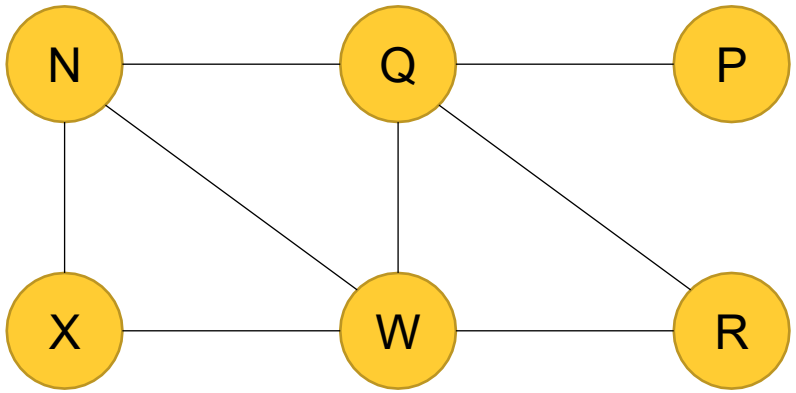
H has no previously unvisited vertices, so we dequeue H which leaves the queue empty, thus indicating that the graph traversal Is complete.



H is dequeued

Queue: empty



Visited: **A B C D E F G H**

In-class: BFS Graph